Stage 6: Starting execution of the initial thread

At this point, the process environment has been determined, resources for its threads to use have been allocated, the process has a thread, and the Windows subsystem knows about the new process. Unless the caller specified the `CREATE_SUSPENDED` flag, the initial thread is now resumed so that it can start running and perform the remainder of the process-initialization work that occurs in the context of the new process (stage 7).

Stage 7: Performing process initialization in the context of the new process

The new thread begins life running the kernel-mode thread startup routine `KiStartUserThread`. `KiStartUserThread` lowers the thread's IRQL level from deferred procedure call (DPC) level to APC level and then calls the system initial thread routine, `PspUserThreadStartup`. The user-specified thread start address is passed as a parameter to this routine. `PspUserThreadStartup` performs the following actions:

1. It installs an exception chain on x86 architecture. (Other architectures work differently in this regard, see Chapter 8 in Part 2.)

2. It lowers IRQL to `PASSIVE_LEVEL` (`0`, which is the only IRQL user code is allowed to run at).

3. It disables the ability to swap the primary process token at runtime.

4. If the thread was killed on startup (for whatever reason), it's terminated and no further action is taken.

5. It sets the locale ID and the ideal processor in the TEB, based on the information present in kernel-mode data structures, and then it checks whether thread creation actually failed.

6. It calls `DbgkCreateThread`, which checks whether image notifications were sent for the new process. If they weren't, and notifications are enabled, an image notification is sent first for the process and then for the image load of Ntdll.dll.

NOTE

This is done in this stage rather than when the images were first mapped because the process ID (which is required for the kernel callouts) is not yet allocated at that time.

7. Once those checks are completed, another check is performed to see whether the process is a debuggee. If it is and if debugger notifications have not been sent yet, then a create process message is sent through the debug object (if one is present) so that the process startup debug event (`CREATE_PROCESS_DEBUG_INFO`) can be sent to the appropriate debugger process. This is followed by a similar thread startup debug event and by another debug event for the image load of Ntdll.dll. `DbgkCreateThread` then waits for a reply from the debugger (via the `Continue- DebugEvent` function).

8. It checks whether application prefetching is enabled on the system and, if so, calls the prefetcher (and Superfetch) to process the prefetch instruction file (if it exists) and prefetch pages referenced during the first 10 seconds the last time the process ran. (For details on the prefetcher and Superfetch, see **Chapter 5**.)

9. It checks whether the system-wide cookie in the `SharedUserData` structure has been set up. If it hasn't, it generates it based on a hash of system information such as the number of interrupts processed, DPC deliveries, page faults, interrupt time, and a random number. This system-wide cookie is used in the internal decoding and encoding of pointers, such as in the heap manager to protect against certain classes of exploitation. (For more information on the heap manager security, see **Chapter 5**.)

10. If the process is secure (IUM process), then a call is made to `HvlStartSecureThread` that transfers control to the secure kernel to start thread execution. This function only returns when the thread exits.

11. It sets up the initial thunk context to run the image-loader initialization routine (`LdrInitialize-Thunk` in Ntdll.dll), as well as the system-wide thread startup stub (`RtlUserThreadStart` in Ntdll.dll). These steps are done by editing the context of the thread in place and then issuing an exit from system service operation, which loads the specially crafted user context. The `LdrInitializeThunk` routine initializes the loader, the heap

manager, NLS tables, thread-local storage (TLS) and fiber-local storage (FLS) arrays, and critical section structures. It then loads any required DLLs and calls the DLL entry points with the `DLL_PROCESS_ATTACH` function code.

Once the function returns, `NtContinue` restores the new user context and returns to user mode. Thread execution now truly starts.

`RtlUserThreadStart` uses the address of the actual image entry point and the start parameter and calls the application's entry point. These two parameters have also already been pushed onto the stack by the kernel. This complicated series of events has two purposes:

- It allows the image loader inside Ntdll.dll to set up the process internally and behind the scenes so that other user-mode code can run properly. (Otherwise, it would have no heap, no thread-local storage, and so on.)

- Having all threads begin in a common routine allows them to be wrapped in exception handling so that if they crash, Ntdll.dll is aware of that and can call the unhandled exception filter inside Kernel32.dll. It is also able to coordinate thread exit on return from the thread's start routine and to perform various cleanup work. Application developers can also call `SetUnhandled- ExceptionFilter` to add their own unhandled exception-handling code.

EXPERIMENT: TRACING PROCESS STARTUP

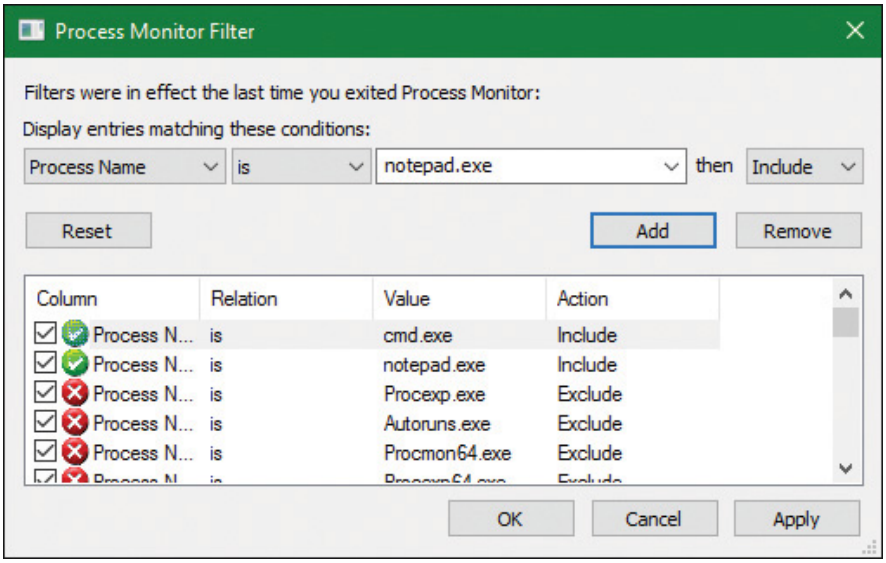
Now that we've looked in detail at how a process starts up and the different operations required to begin executing an application, we're going to use Process Monitor to look at some of the file I/O and registry keys that are accessed during this process.

Although this experiment will not provide a complete picture of all the internal steps we've described, you'll be able to see several parts of the system in action, notably prefetch and Superfetch, image-file execution options and other compatibility checks, and the image loader's DLL mapping.

We'll look at a very simple executable—Notepad.exe—and launch it from a Command Prompt window (Cmd.exe). It's important that we look both at the operations inside Cmd.exe and those inside Notepad.exe. Recall that a lot of the user-mode work is performed by `CreateProcessInternalW`, which is called by the parent process before the kernel has created a new process object.

To set things up correctly, follow these steps:

1. Add two filters to Process Monitor: one for Cmd.exe and one for Notepad.exe. These are the only two processes you should include. Be sure you don't have any currently running instances of these two processes so that you know you're looking at the right events. The filter window should look like this:

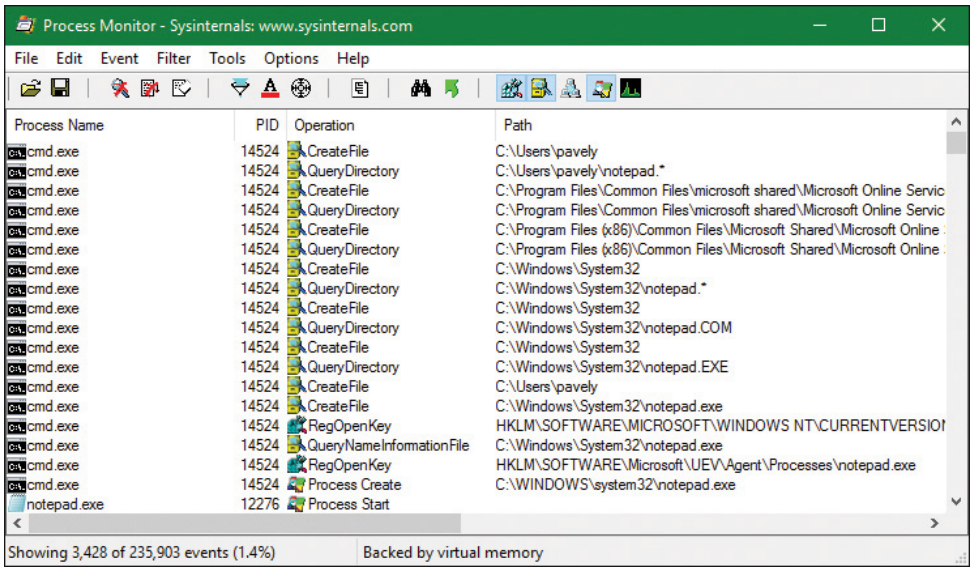


2. Make sure event logging is currently disabled (open the **File** and deselect **Capture Events**) and then start the command prompt.

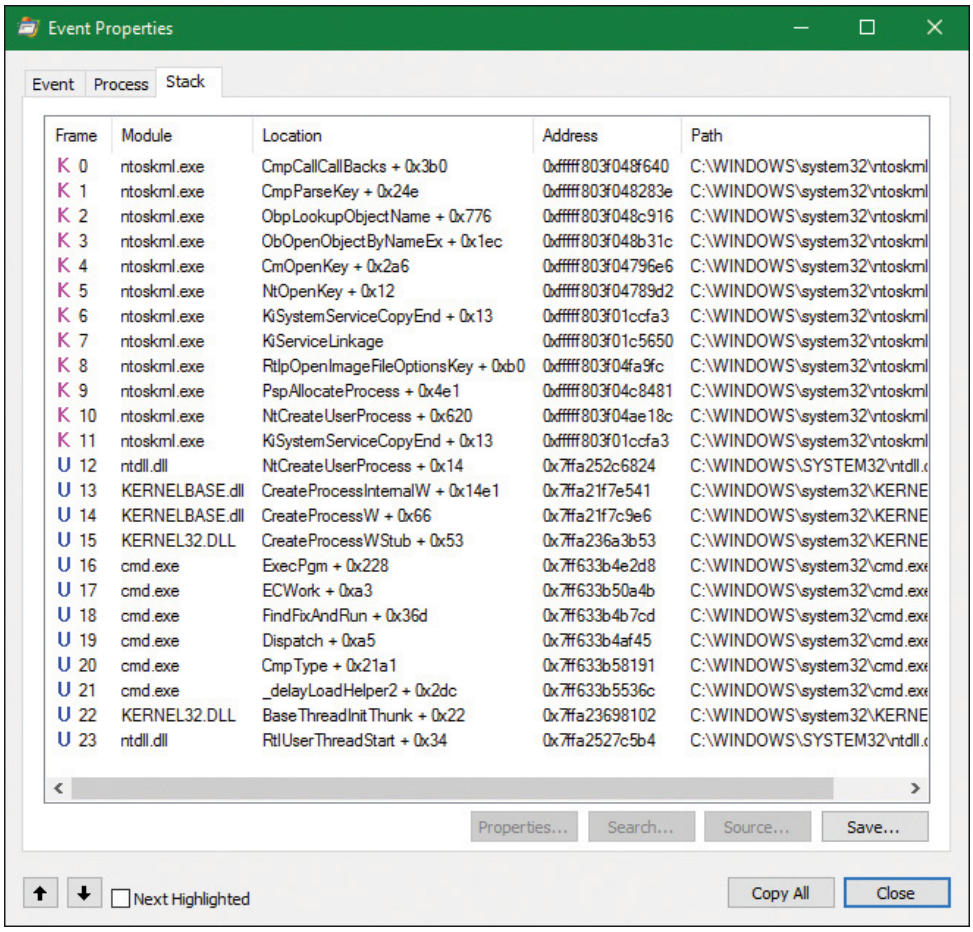
3. Enable event logging (open the **File** menu and choose **Event Logging**, press **Ctrl+E**, or click the magnifying glass icon on the toolbar) and then type **Notepad.exe** and press **Enter**. On a typical Windows system, you should see anywhere between 500 and 3,500 events appear.

4. Stop capture and hide the Sequence and Time of Day columns so that you can focus your attention on the columns of interest. Your window should look similar to the one shown in the following screenshot.

As described in stage 1 of the `CreateProcess` flow, one of the first things to notice is that just before the process is started and the first thread is created, Cmd.exe does a registry read at HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\Notepad.exe. Because there were no image-execution options associated with Notepad.exe, the process was created as is.



As with this and any other event in Process Monitor's log, you can see whether each part of the process-creation flow was performed in user mode or kernel mode, and by which routines, by looking at the stack of the event. To do this, double-click the RegOpenKey event and switch to the **Stack** tab. The following screenshot shows the standard stack on a 64-bit Windows 10 machine:



This stack shows that you already reached the part of process creation performed in kernel mode (through `NtCreateUserProcess`) and that the helper routine `PspAllocateProcess` is responsible for this check.

Going down the list of events after the thread and process have been created, you will notice three groups of events:

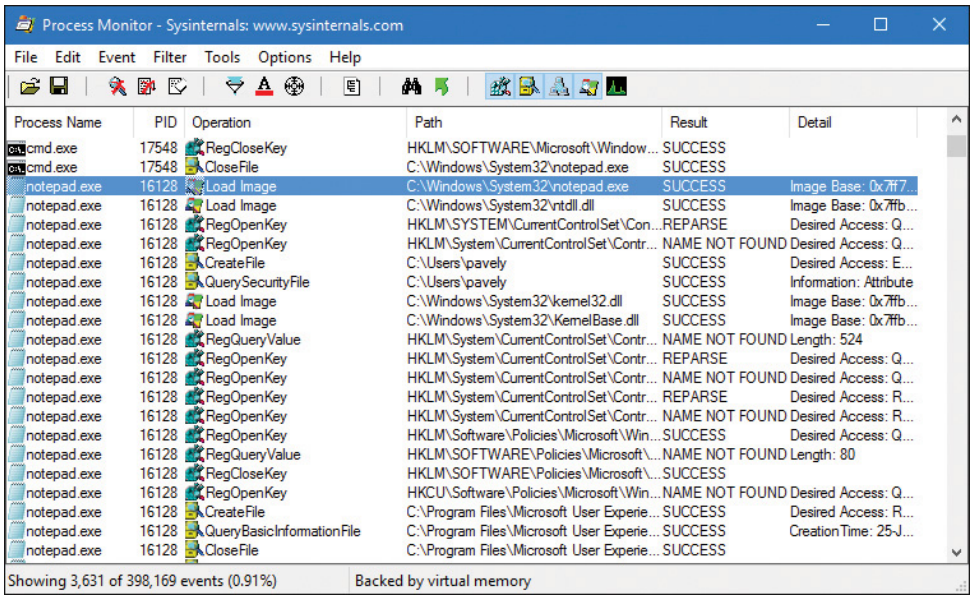
- A simple check for application-compatibility flags, which will let the user-mode process-creation code know if checks inside the application-compatibility database are required through the shim engine.

- Multiple reads to SxS (search for Side-By-Side), Manifest, and MUI/Language keys, which are part of the assembly framework mentioned earlier.

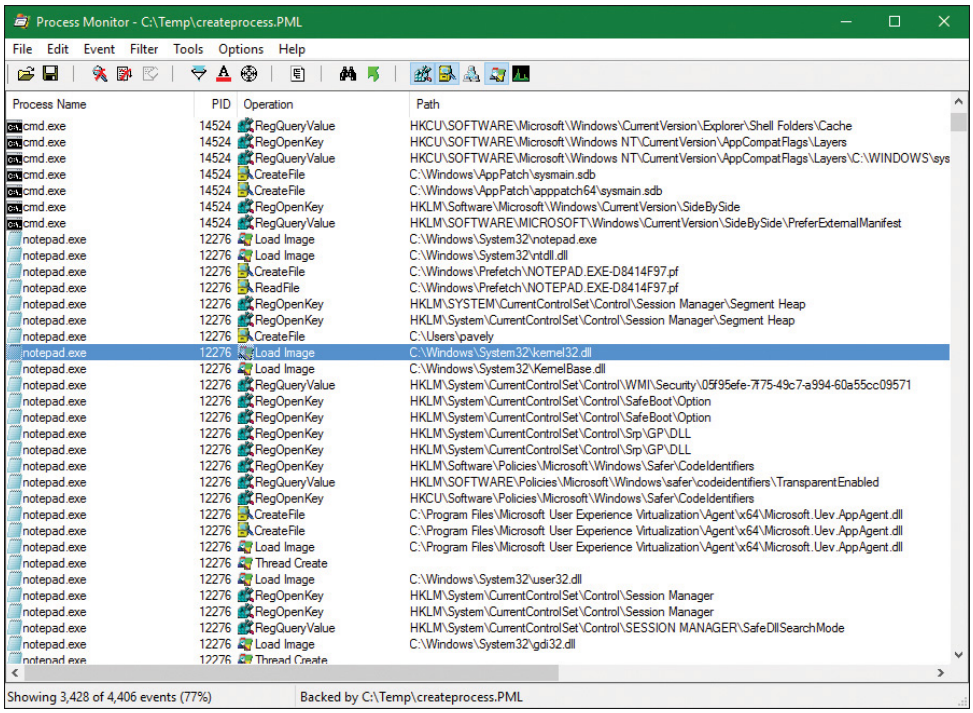
- File I/O to one or more .sdb files, which are the application-compatibility databases on the system. This I/O is where additional checks are done to see if the shim engine needs to be invoked for this application. Because Notepad is a well-behaved Microsoft program, it doesn't require any shims.

The following screenshot shows the next series of events, which happen inside the Notepad process itself. These are actions initiated by the user-

mode thread startup wrapper in kernel mode, which performs the actions described earlier. The first two are the Notepad.exe and Ntdll.dll image load debug notification messages, which can be generated only now that code is running inside Notepad's process context and not inside the context for the command prompt.

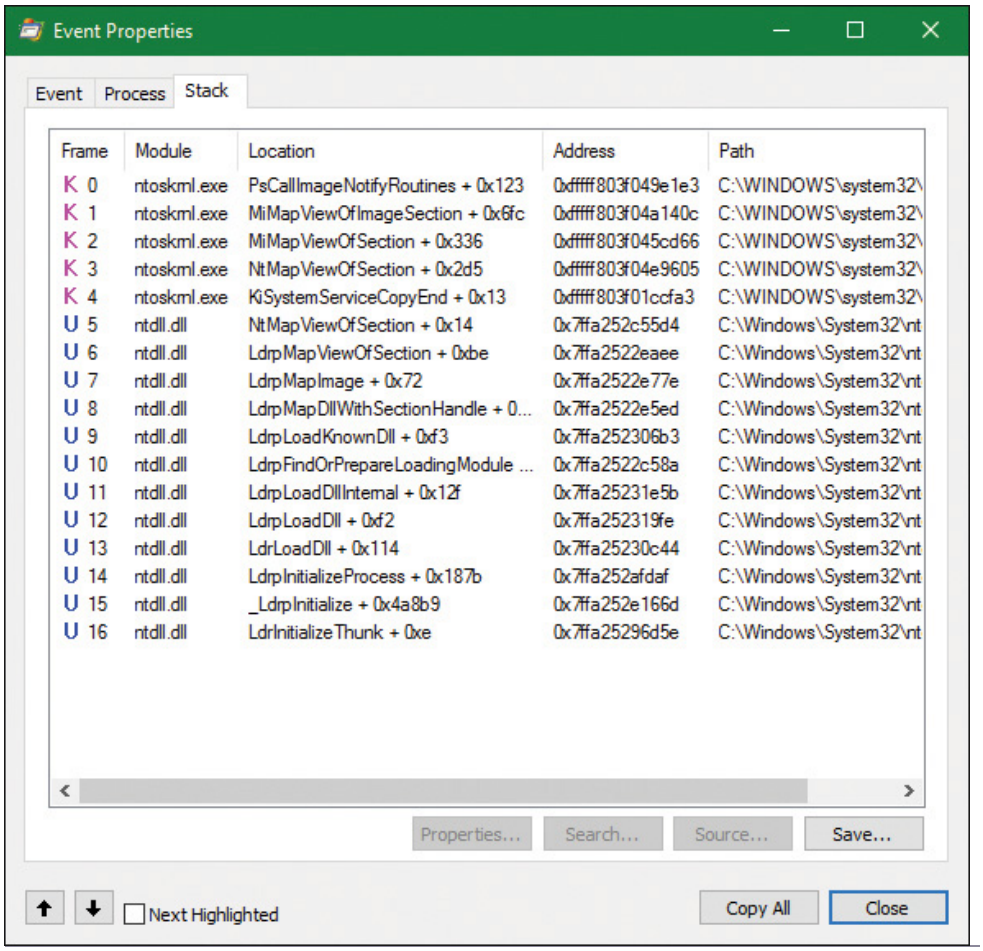


Next, the prefetcher kicks in, looking for a prefetch database file that has already been generated for Notepad. (For more information on the prefetcher, see **Chapter 5**.) On a system where Notepad has already been run at least once, this database will exist, and the prefetcher will begin executing the commands specified inside it. If this is the case, scrolling down, you will see multiple DLLs being read and queried. Unlike typical DLL loading, which is done by the user-mode image loader by looking at the import tables or when an application manually loads a DLL, these events are being generated by the prefetcher, which is already aware of the libraries that Notepad will require. Typical image loading of the DLLs required happens next, and you will see events similar to the ones shown here:



These events are now being generated from code running inside user mode, which was called once the kernel-mode wrapper function finished its work. Therefore, these are the first events coming from `LdrpInitializeProcess`, which is called by `LdrInitializeThunk` for the first thread in the process. You can confirm this on your own by looking at the stack of these events—for example, the kernel32.dll image load event, which is shown in the following screenshot.

Further events are generated by this routine and its associated helper functions until you finally reach events generated by the `WinMain` function inside Notepad, which is where code under the developer's control is now being executed. Describing in detail all the events and user-mode components that come into play during process execution would fill up this entire chapter, so exploration of any further events is left as an exercise for the reader.



Terminating a process

A process is a container and a boundary. This means resources used by one process are not automatically visible in other processes, so some inter-process communication mechanism needs to be used to pass information between processes. Therefore, a process cannot accidentally write arbitrary bytes on another process's memory. That would require explicit call to a function such as `WriteProcessMemory`. However, to get that to work, a handle with the proper access mask (`PROCESS_VM_WRITE`) must be opened explicitly, which may or may not be granted. This natural isolation between processes also means that if some exception happens in one process, it will have no effect on other processes. The worst that can happen is that same process would crash, but the rest of the system stays intact.

A process can exit gracefully by calling the `ExitProcess` function. For many processes—depending on linker settings—the process startup code for the first thread calls `ExitProcess` on the process's behalf when the thread returns from its main function. The term *gracefully* means that DLLs loaded into the process get a chance to do some work by getting no-

tified of the process exit using a call to their `DllMain` function with `DLL_PROCESS_DETACH`.

`ExitProcess` can be called only by the process itself asking to exit. An ungraceful termination of a process is possible using the `TerminateProcess` function, which can be called from outside the process. (For example, Process Explorer and Task Manager use it when so requested.) `TerminateProcess` requires a handle to the process that is opened with the `PROCESS_TERMINATE` access mask, which may or may not be granted. This is why it's not easy (or it's impossible) to terminate some processes (for example, Csrss)—the handle with the required access mask cannot be obtained by the requesting user. The meaning of *ungraceful* here is that DLLs don't get a chance to execute code (`DLL_PROCESS_DETACH` is not sent) and all threads are terminated abruptly. This can lead to data loss in some cases—for example, if a file cache has no chance to flush its data to the target file.

In whatever way a process ceases to exist, there can never be any leaks. That is, all process's private memory is freed automatically by the kernel, the address space is destroyed, all handles to kernel objects are closed, etc. If open handles to the process still exist (the `EPROCESS` structure still exists), then other processes can still gain access to some process-management information, such as the process exit code (`GetExitCodeProcess`). Once these handles are closed, the `EPROCESS` is properly destroyed, and there's truly nothing left of the process.

That being said, if third party drivers make allocations in kernel memory on behalf of a process—say, due to an IOCTL or merely due to a process notification—it is their responsibility to free any such pool memory on their own. Windows does not track or clean-up process-owned kernel memory (except for memory occupied by objects due to handles that the process created). This would typically be done through the `IRP_MJ_CLOSE` or `IRP_MJ_CLEANUP` notification to tell the driver that the handle to the device object has been closed, or through a process termination notification. (see **Chapter 6**, "**I/O system**," for more on IOCTLs.)

Image loader

As we've just seen, when a process is started on the system, the kernel creates a process object to represent it and performs various kernel-related initialization tasks. However, these tasks do not result in the execution of the application, merely in the preparation of its context and environment. In fact, unlike drivers, which are kernel-mode code, applications execute in user mode. So most of the actual initialization work is done outside the kernel. This work is performed by the *image loader*, also internally referred to as `Ldr`.

The image loader lives in the user-mode system DLL *Ntdll.dll* and not in the kernel library. Therefore, it behaves just like standard code that is part of a DLL, and it is subject to the same restrictions in terms of memory access and security rights. What makes this code special is the guarantee that it will always be present in the running process (Ntdll.dll is always loaded) and that it is the first piece of code to run in user mode as part of a new process.

Because the loader runs before the actual application code, it is usually invisible to users and developers. Additionally, although the loader's initialization tasks are hidden, a program typically does interact with its interfaces during the run time of a program—for example, whenever loading or unloading a DLL or querying the base address of one. Some of the main tasks the loader is responsible for include:

■ Initializing the user-mode state for the application, such as creating the initial heap and setting up the thread-local storage (TLS) and fiber-local storage (FLS) slots.

■ Parsing the import table (IAT) of the application to look for all DLLs that it requires (and then recursively parsing the IAT of each DLL), followed by parsing the export table of the DLLs to make sure the function is actually present. (Special *forwarder entries* can also redirect an export to yet another DLL.)

■ Loading and unloading DLLs at run time, as well as on demand, and maintaining a list of all loaded modules (the module database).

■ Handling manifest files, needed for Windows Side-by-Side (SxS) support, as well as Multiple Language User Interface (MUI) files and resources.

■ Reading the application compatibility database for any shims, and loading the shim engine DLL if required.

■ Enabling support for API Sets and API redirection, a core part of the One Core functionality that allows creating Universal Windows Platform (UWP) applications.

■ Enabling dynamic runtime compatibility mitigations through the *SwitchBack* mechanism as well as interfacing with the shim engine and Application Verifier mechanisms.

As you can see, most of these tasks are critical to enabling an application to actually run its code. Without them, everything from calling external functions to using the heap would immediately fail. After the process has been created, the loader calls the `NtContinue` special native API to continue execution based on an exception frame located on the stack, just as an exception handler would. This exception frame, built by the kernel as we saw in an earlier section, contains the actual entry point of the application. Therefore, because the loader doesn't use a standard call or jump into the running application, you'll never see the loader initialization functions as part of the call tree in a stack trace for a thread.

EXPERIMENT: WATCHING THE IMAGE LOADER

In this experiment, you'll use global flags to enable a debugging feature called *loader snaps*. This allows you to see debug output from the image loader while debugging application startup.

1. From the directory where you've installed WinDbg, launch the Gflags.exe application, and then click the **Image File** tab.

2. In the Image field, type **Notepad.exe**, and then press the **Tab** key. This should enable the various options. Select the **Show Loader Snaps** option and then click **OK** or **Apply**.

3. Now launch WinDbg, open the **File** menu, choose **Open Executable**, and navigate to c:\windows\system32\notepad.exe to launch it. You should see a couple of screens of debug information similar to that shown here:

Click here to view code image

```
0f64:2090 @ 02405218 - LdrpInitializeProcess - INFO: Beginning execution
of
notepad.exe (C:\WINDOWS\notepad.exe)
   Current directory: C:\Program Files (x86)\Windows Kits\10\Debuggers\
   Package directories: (null)
0f64:2090 @ 02405218 - LdrLoadDll - ENTER: DLL name: KERNEL32.DLL
0f64:2090 @ 02405218 - LdrpLoadDllInternal - ENTER: DLL name:
KERNEL32.DLL
0f64:2090 @ 02405218 - LdrpFindKnownDll - ENTER: DLL name:
KERNEL32.DLL
0f64:2090 @ 02405218 - LdrpFindKnownDll - RETURN: Status: 0x00000000
0f64:2090 @ 02405218 - LdrpMinimalMapModule - ENTER: DLL name:
C:\WINDOWS\System32\KERNEL32.DLL
ModLoad: 00007fff'5b4b0000
00007fff'5b55d000   C:\WINDOWS\System32\KERNEL32.DLL
0f64:2090 @ 02405218 - LdrpMinimalMapModule - RETURN: Status:
0x00000000
0f64:2090 @ 02405218 - LdrpPreprocessDllName - INFO: DLL api-ms-win-
core-
rtlsupport-l1-2-0.dll was redirected to C:\WINDOWS\SYSTEM32\ntdll.dll by
API set
0f64:2090 @ 02405218 - LdrpFindKnownDll - ENTER: DLL name:
KERNELBASE.dll
```

0f64:2090 @ 02405218 - LdrpFindKnownDll - RETURN: Status: 0x00000000
0f64:2090 @ 02405218 - LdrpMinimalMapModule - ENTER: DLL name: C:\WINDOWS\
System32\KERNELBASE.dll
ModLoad: 00007fff'58b90000
00007fff'58dc6000   C:\WINDOWS\System32\KERNELBASE.dll
0f64:2090 @ 02405218 - LdrpMinimalMapModule - RETURN: Status: 0x00000000
0f64:2090 @ 02405218 - LdrpPreprocessDllName - INFO: DLL api-ms-win-eventing-provider-l1-1-0.dll was redirected to C:\WINDOWS\SYSTEM32\kernelbase.dll by API set
0f64:2090 @ 02405218 - LdrpPreprocessDllName - INFO: DLL api-ms-win-core-
apiquery-l1-1-0.dll was redirected to C:\WINDOWS\SYSTEM32\ntdll.dll by API set

4. Eventually, the debugger breaks somewhere inside the loader code, at a location where the image loader checks whether a debugger is attached and fires a breakpoint. If you press the **g** key to continue execution, you will see more messages from the loader, and Notepad will appear.

5. Try interacting with Notepad and see how certain operations invoke the loader. A good experiment is to open the Save/Open dialog box. That demonstrates that the loader not only runs at startup, but continuously responds to thread requests that can cause *delayed loads* of other modules (which can then be unloaded after use).

Early process initialization

Because the loader is present in Ntdll.dll, which is a native DLL that's not associated with any particular subsystem, all processes are subject to the same loader behavior (with some minor differences). Earlier, we took a detailed look at the steps that lead to the creation of a process in kernel mode, as well as some of the work performed by the Windows function `CreateProcess`. Here, we'll cover all the *other* work that takes place in user mode, independent of any subsystem, as soon as the first user-mode instruction starts execution.

When a process starts, the loader performs the following steps:

1. It checks if `LdrpProcessInitialized` is already set to `1` or if the `SkipLoaderInit` flag is set in the TEB. In this case, skip all initialization and wait three seconds for someone to call `LdrpProcess-InitializationComplete`. This is used in cases where process reflection is used by Windows Error Reporting, or other process fork attempts where loader initialization is not needed.

2. It sets the `LdrInitState` to `0`, meaning that the loader is uninitialized. Also set the PEB's `ProcessInitializing` flag to `1` and the TEB's `RanProcessInit` to `1`.

3. It initializes the loader lock in the PEB.

4. It initializes the dynamic function table, used for unwind/exception support in JIT code.

5. It initializes the Mutable Read Only Heap Section (MRDATA), which is used to store security- relevant global variables that should not be modified by exploits (see **Chapter 7** for more information).

6. It initializes the loader database in the PEB.

7. It initializes the National Language Support (NLS, for internationalization) tables for the process.

8. It builds the image path name for the application.

9. It captures the SEH exception handlers from the .pdata section and builds the internal exception tables.

10. It captures the system call thunks for the five critical loader functions: `NtCreateSection`, `NtOpenFile`, `NtQueryAttributesFile`, `NtOpenSection`, and `NtMapViewOfSection.`

11. It reads the mitigation options for the application (which are passed in by the kernel through the `LdrSystemDllInitBlock` exported variable). These are described in more detail in **Chapter 7**.

12. It queries the Image File Execution Options (IFEO) registry key for the application. This will include options such as the global flags (stored in `GlobalFlags`), as well as heap-debugging options (`DisableHeapLookaside`, `ShutdownFlags`, and `FrontEndHeapDebugOptions`), loader settings (`UnloadEventTraceDepth`,

`MaxLoaderThreads`, `UseImpersonatedDeviceMap`), ETW settings (`TracingFlags`). Other options include `MinimumStackCommitInBytes` and `MaxDeadActivationContexts`. As part of this work, the Application Verifier package and related Verifier DLLs will be initialized and Control Flow Guard (CFG) options will be read from `CFGOptions`.

13. It looks inside the executable's header to see whether it is a .NET application (specified by the presence of a .NET-specific image directory) and if it's a 32-bit image. It also queries the kernel to verify if this is a Wow64 process. If needed, it handles a 32-bit IL-only image, which does not require Wow64.

14. It loads any configuration options specified in the executable's Image Load Configuration Directory. These options, which a developer can define when compiling the application, and which the compiler and linker also use to implement certain security and mitigation features such as CFG, control the behavior of the executable.

15. It minimally initializes FLS and TLS.

16. It sets up debugging options for critical sections, creates the user-mode stack trace database if the appropriate global flag was enabled, and queries `StrackTraceDatabaseSizeInMb` from the Image File Execution Options.

17. It initializes the heap manager for the process and creates the first process heap. This will use various load configuration, image file execution, global flags, and executable header options to set up the required parameters.

18. It enables the Terminate process on heap corruption mitigation if it's turned on.

19. It initializes the exception dispatch log if the appropriate global flag has enabled this.

20. It initializes the thread pool package, which supports the Thread Pool API. This queries and takes into account NUMA information.

21. It initializes and converts the environment block and parameter block, especially as needed to support WoW64 processes.

22. It opens the \KnownDlls object directory and builds the known DLL path. For a Wow64 process, \KnownDlls32 is used instead.

23. For store applications, it reads the Application Model Policy options, which are encoded in the WIN://PKG and WP://SKUID claims of the token (see the "**AppContainers**" section in **Chapter 7** for more information).

24. It determines the process's current directory, system path, and default load path (used when loading images and opening files), as well as the rules around default DLL search order. This includes reading the current policy settings for Universal (UWP) versus Desktop Bridge (Centennial) versus Silverlight (Windows Phone 8) packaged applications (or services).

25. It builds the first loader data table entry for Ntdll.dll and inserts it into the module database.

26. It builds the unwind history table.

27. It initializes the parallel loader, which is used to load all the dependencies (which don't have cross-dependencies) using the thread pool and concurrent threads.

28. It builds the next loader data table entry for the main executable and inserts it into the module database.

29. If needed, it relocates the main executable image.

30. If enabled, it initializes Application Verifier.

31. It initializes the Wow64 engine if this is a Wow64 process. In this case, the 64-bit loader will finish its initialization, and the 32-bit loader will take control and re-start most of the operations we've just described up until this point.

32. If this is a .NET image, it validates it, loads Mscoree.dll (.NET runtime shim), and retrieves the main executable entry point (`_CorExeMain`), overwriting the exception record to set this as the entry point instead of the regular main function.

33. It initializes the TLS slots of the process.

34. For Windows subsystem applications, it manually loads Kernel32.dll and Kernelbase.dll, regardless of actual imports of the process. As needed,

it uses these libraries to initialize the SRP/Safer (Software Restriction Policies) mechanisms, as well as capture the Windows subsystem thread initialization thunk function. Finally, it resolves any API Set dependencies that exist specifically between these two libraries.

35. It initializes the shim engine and parses the shim database.

36. It enables the parallel image loader, as long as the core loader functions scanned earlier do not have any system call hooks or "detours" attached to them, and based on the number of loader threads that have been configured through policy and image file execution options.

37. It sets the `LdrInitState` variable to `1`, meaning "import loading in progress."

At this point, the image loader is ready to start parsing the import table of the executable belonging to the application and start loading any DLLs that were dynamically linked during the compilation of the application. This will happen both for .NET images, which will have their imports processed by calling into the .NET runtime, as well as for regular images. Because each imported DLL can also have its own import table, this operation, in the past, continued recursively until all DLLs had been satisfied and all functions to be imported have been found. As each DLL was loaded, the loader kept state information for it and built the module database.

In newer versions of Windows, the loader instead builds a dependency map ahead of time, with specific nodes that describe a single DLL and its dependency graph, building out separate nodes that can be loaded in parallel. At various points when serialization is needed, the thread pool worker queue is "drained," which services as a synchronization point. One such point is before calling all the DLL initialization routines of all the static imports, which is one of the last stages of the loader. Once this is done, all the static TLS initializers are called. Finally, for Windows applications, in between these two steps, the Kernel32 thread initialization thunk function (`BaseThreadInitThunk`) is called at the beginning, and the Kernel32 post-process initialization routine is called at the end.

DLL name resolution and redirection

Name resolution is the process by which the system converts the name of a PE-format binary to a physical file in situations where the caller has not specified or cannot specify a unique file identity. Because the locations of various directories (the application directory, the system directory, and so on) cannot be hardcoded at link time, this includes the resolution of all binary dependencies as well as `LoadLibrary` operations in which the caller does not specify a full path.

When resolving binary dependencies, the basic Windows application model locates files in a search path—a list of locations that is searched sequentially for a file with a matching base name—although various system components override the search path mechanism in order to extend the default application model. The notion of a search path is a holdover from the era of the command line, when an application's current directory was a meaningful notion; this is somewhat anachronistic for modern GUI applications.

However, the placement of the current directory in this ordering allowed load operations on system binaries to be overridden by placing malicious binaries with the same base name in the application's current directory, a technique often known as *binary planting*. To prevent security risks associated with this behavior, a feature known as *safe DLL search mode* was added to the path search computation and is enabled by default for all processes. Under safe search mode, the current directory is moved behind the three system directories, resulting in the following path ordering:

1. The directory from which the application was launched

2. The native Windows system directory (for example, C:\Windows\System32)

3. The 16-bit Windows system directory (for example, C:\Windows\System)

4. The Windows directory (for example, C:\Windows)

5. The current directory at application launch time

6. Any directories specified by the %PATH% environment variable

The DLL search path is recomputed for each subsequent DLL load operation. The algorithm used to compute the search path is the same as the one used to compute the default search path, but the application can change specific path elements by editing the %PATH% variable using the `SetEnvironmentVariable` API, changing the current directory using the `SetCurrentDirectory` API, or using the `SetDllDirectory` API to specify a DLL directory for the process. When a DLL directory is specified, the directory replaces the current directory in the search path and the loader ignores the safe DLL search mode setting for the process.

Callers can also modify the DLL search path for specific load operations by supplying the `LOAD_WITH_ALTERED_SEARCH_PATH` flag to the `LoadLibraryEx` API. When this flag is supplied and the DLL name supplied to the API specifies a full path string, the path containing the DLL file is used in place of the application directory when computing the search path for the operation. Note that if the path is a relative path, this behavior is undefined and potentially dangerous. When Desktop Bridge (Centennial) applications load, this flag is ignored.

Other flags that applications can specify to `LoadLibraryEx` include `LOAD_LIBRARY_SEARCH_DLL_LOAD_DIR`, `LOAD_LIBRARY_SEARCH_APPLICATION_DIR`, `LOAD_LIBRARY_SEARCH_SYSTEM32`, and `LOAD_LIBRARY_SEARCH_USER_DIRS`, in place of the `LOAD_WITH_ALTERED_SEARCH_PATH` flag. Each of these modifies the search order to only search the specific directory (or directories) that the flag references, or the flags can be combined as desired to search multiple locations. For example, combining the application, system32, and user directories results in `LOAD_LIBRARY_SEARCH_DEFAULT_DIRS`. Furthermore, these flags can be globally set using the `SetDefaultDllDirectories` API, which will affect all library loads from that point on.

Another way search-path ordering can be affected is if the application is a packaged application or if it is not a packaged service or legacy Silverlight 8.0 Windows Phone application. In these conditions, the DLL search order will not use the traditional mechanism and APIs, but will rather be restricted to the package-based graph search. This is also the case when the `LoadPackagedLibrary` API is used instead of the regular `LoadLibraryEx` function. The package-based graph is computed based on the `<PackageDependency>` entries in the UWP application's manifest file's `<Dependencies>` section, and guarantees that no arbitrary DLLs can accidentally load in the package.

Additionally, when a packaged application is loaded, as long as it is not a Desktop Bridge application, all application-configurable DLL search path ordering APIs, such as the ones we saw earlier, will be disabled, and only the default system behavior will be used (in combination with only looking through package dependencies for most UWP applications as per the above).

Unfortunately, even with safe search mode and the default path searching algorithms for legacy applications, which always include the application directory first, a binary might still be copied from its usual location to a user-accessible location (for example, from c:\windows\system32\notepad.exe into c:\temp\notepad.exe, an operation that does not require administrative rights). In this situation, an attacker can place a specifically crafted DLL in the same directory as the application, and due to the ordering above, it will take precedence over the system DLL. This can then be used for persistence or otherwise affecting the application, which might be privileged (especially if the user, unaware of the change, is elevating it through UAC). To defend against this, processes and/or administrators can use a process-mitigation policy (see **Chapter 7** for more information on these) called Prefer System32 Images, which inverts the order above between points 1 and 2, as the name suggests.

DLL name redirection

Before attempting to resolve a DLL name string to a file, the loader attempts to apply DLL name redirection rules. These redirection rules are used to extend or override portions of the DLL namespace—which normally corresponds to the Win32 file system namespace—to extend the Windows application model. In order of application, these are:

- **MinWin API Set redirection** The API set mechanism is designed to allow different versions or editions of Windows to change the binary that exports a given system API in a manner that is transparent to applications, by introducing the concept of contracts. This mechanism was briefly touched upon in **Chapter 2**, and will be further explained in a later section.

- **.LOCAL redirection** The .LOCAL redirection mechanism allows applications to redirect all loads of a specific DLL base name, regardless of whether a full path is specified, to a local copy of the DLL in the application directory—either by creating a copy of the DLL with the same base name followed by *.local* (for example, MyLibrary.dll.local) or by creating

a file folder with the name .local under the application directory and placing a copy of the local DLL in the folder (for example, C:\\MyApp\.LOCAL\MyLibrary.dll). DLLs redirected by the .LOCAL mechanism are handled identically to those redirected by SxS. (See the next bullet point.) The loader honors .LOCAL redirection of DLLs only when the executable does not have an associated manifest, either embedded or external. It's not enabled by default. To enable it globally, add the DWORD value `DevOverrideEnable` in the base IFEO key (HKLM\Software\Microsoft\WindowsNT\CurrentVersion\Image File Execution Options) and set it to `1`.

■ **Fusion (SxS) redirection** Fusion (also referred to as *side-by-side, or SxS*) is an extension to the Windows application model that allows components to express more detailed binary dependency information (usually versioning information) by embedding binary resources known as *manifests*. The Fusion mechanism was first used so that applications could load the correct version of the Windows common controls package (comctl32.dll) after that binary was split into different versions that could be installed alongside one another; other binaries have since been versioned in the same fashion. As of Visual Studio 2005, applications built with the Microsoft linker use Fusion to locate the appropriate version of the C runtime libraries, while Visual Studio 2015 and later use API Set redirection to implement the idea of the universal CRT.

The Fusion runtime tool reads embedded dependency information from a binary's resource section using the Windows resource loader, and it packages the dependency information into lookup structures known as *activation contexts*. The system creates default activation contexts at the system and process level at boot and process startup time, respectively; in addition, each thread has an associated activation context stack, with the activation context structure at the top of the stack considered active. The per-thread activation context stack is managed both explicitly, via the `ActivateActCtx` and `DeactivateActCtx` APIs, and implicitly by the system at certain points, such as when the DLL main routine of a binary with embedded dependency information is called. When a Fusion DLL name redirection lookup occurs, the system searches for redirection information in the activation context at the head of the thread's activation context stack, followed by the process and system activation contexts; if redirection information is present, the file identity specified by the activation context is used for the load operation.

■ **Known DLL redirection** Known DLLs is a mechanism that maps specific DLL base names to files in the system directory, preventing the DLL from being replaced with an alternate version in a different location.

One edge case in the DLL path search algorithm is the DLL versioning check performed on 64-bit and WoW64 applications. If a DLL with a matching base name is located but is subsequently determined to have been compiled for the wrong machine architecture—for example, a 64-bit image in a 32-bit application—the loader ignores the error and resumes the path search operation, starting with the path element after the one used to locate the incorrect file. This behavior is designed to allow applications to specify both 64-bit and 32-bit entries in the global %PATH% environment variable.

EXPERIMENT: OBSERVING DLL LOAD SEARCH ORDER

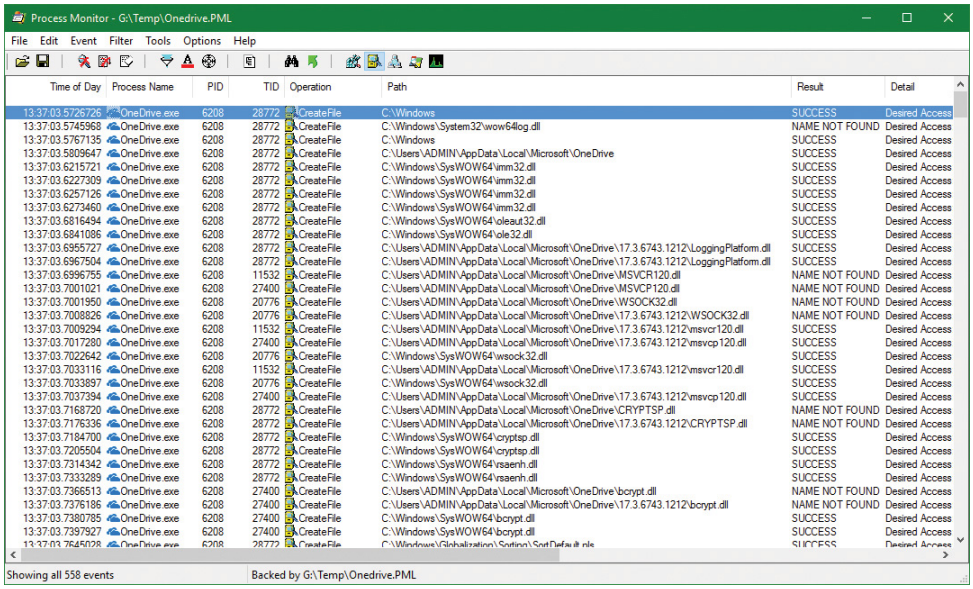
You can use Sysinternals Process Monitor tool to watch how the loader searches for DLLs. When the loader attempts to resolve a DLL dependency, you will see it perform `CreateFile` calls to probe each location in the search sequence until either it finds the specified DLL or the load fails.

Here's the capture of the loader's search for the OneDrive.exe executable. To re-create the experiment, do the following:

1. If the OneDrive is running, close it from its tray icon. Make sure to close all Explorer windows that are looking at OneDrive content.

2. Open Process Monitor and add filters to show just the process OneDrive.exe. Optionally, show only the operation for `CreateFile`.

3. Go to %LocalAppData%\Microsoft\OneDrive and launch OneDrive.exe or OneDrive Personal.cmd (which launches OneDrive.exe as "personal" rather than "business"). You should see something like the following (note that OneDrive is a 32 bit process, here running on a 64 bit system):



Here are some of the calls shown as they relate to the search order described previously:

■ KnownDlls DLLs load from the system location (ole32.dll in the screenshot).

■ LoggingPlatform.Dll is loaded from a version subdirectory, probably because OneDrive calls SetDllDirectory to redirect searches to the latest version (17.3.6743.1212 in the screenshot).

■ The MSVCR120.dll (MSVC run time version 12) is searched for in the executable's directory, and is not found. Then it's searched in the version subdirectory, where it's located.

■ The Wsock32.Dll (WinSock) is searched in the executable's path, then in the version subdirectory, and finally located in the system directory (SysWow64). Note that this DLL is not a KnownDll.

Loaded module database

The loader maintains a list of all modules (DLLs as well as the primary executable) that have been loaded by a process. This information is stored in the PEB—namely, in a substructure identified by *Ldr* and called `PEB_LDR_DATA`. In the structure, the loader maintains three doubly linked lists, all containing the same information but ordered differently (either by load order, memory location, or initialization order). These lists contain structures called *loader data table entries* (`LDR_DATA_TABLE_ENTRY`) that store information about each module.

Additionally, because lookups in linked lists are algorithmically expensive (being done in linear time), the loader also maintains two red-black trees, which are efficient binary lookup trees. The first is sorted by base address, while the second is sorted by the hash of the module's name. With these trees, the searching algorithm can run in logarithmic time, which is significantly more efficient and greatly speeds up process-creation performance in Windows 8 and later. Additionally, as a security precaution, the root of these two trees, unlike the linked lists, is not accessible in the PEB. This makes them harder to locate by shell code, which is operating in an environment where address space layout randomization (ASLR) is enabled. (See **Chapter 5** for more on ASLR.)

Table 3-9 lists the various pieces of information the loader maintains in an entry.

Field	Meaning
BaseAddressIndexNode	Links this entry as a node in the Red-Black Tree sorted by base address.
BaseDllName/ BaseNameHashValue	The name of the module itself, without the full path. The second field stores its hash using RtlHashUnicodeString.
DdagNode/NodeModuleLink	A pointer to the data structure tracking the distributed dependency graph (DDAG), which parallelizes dependency loading through the worker thread pool. The second field links the structure with the LDR_DATA_TABLE_ENTRYs associated with it (part of the same graph).
DllBase	Holds the base address at which the module was loaded.
EntryPoint	Contains the initial routine of the module (such as DllMain).
EntryPointActivationContext	Contains the SxS/Fusion activation context when calling initializers.
Flags	Loader state flags for this module (see Table 3-10 for a description of the flags).
ForwarderLinks	A linked list of modules that were loaded as a result of export table forwarders from the module.
FullDllName	The fully qualified path name of the module.
HashLinks	A linked list used during process startup and shutdown for quicker lookups.
ImplicitPathOptions	Used to store path lookup flags that can be set with the LdrSetImplicitPathOptions API or that are inherited based on the DLL path.
List Entry Links	Links this entry into each of the three ordered lists part of the loader database.
LoadContext	Pointer to the current load information for the DLL. Typically NULL unless actively being loaded.
ObsoleteLoadCount	A reference count for the module (that is, how many times it has been loaded). This is no longer accurate and has been moved to the DDAG node structure instead.
LoadReason	Contains an enumeration value that explains why this DLL was loaded (dynamically, statically, as a forwarder, as a delay-load dependency, etc.).
LoadTime	Stores the system time value when this module was being loaded.
MappingInfoIndexNode	Links this entry as a node in the red-black tree sorted by the hash of the name.
OriginalBase	Stores the original base address (set by the linker) of this module, before ASLR or relocations, enabling faster processing of relocated import entries.
ParentDllBase	In case of static (or forwarder, or delay-load) dependencies, stores the address of the DLL that has a dependency on this one.
SigningLevel	Stores the signature level of this image (see Chapter 8, Part 2, for more information on the Code Integrity infrastructure).
SizeOfImage	The size of the module in memory.
SwitchBackContext	Used by SwitchBack (described later) to store the current Windows context GUID associated with this module, and other data.
TimeDateStamp	A time stamp written by the linker when the module was linked, which the loader obtains from the module's image PE header.
TlsIndex	The thread local storage slot associated with this module.

TABLE 3-9 Fields in a loader data table entry

One way to look at a process's loader database is to use WinDbg and its formatted output of the PEB. The next experiment shows you how to do this and how to look at the `LDR_DATA_TABLE_ENTRY` structures on your own.

EXPERIMENT: DUMPING THE LOADED MODULES DATABASE

Before starting the experiment, perform the same steps as in the previous two experiments to launch Notepad.exe with WinDbg as the debugger. When you get to the initial breakpoint (where you've been instructed to type **g** until now), follow these instructions:

1. You can look at the PEB of the current process with the `!peb` command. For now, you're interested only in the `Ldr` data that will be displayed.

Click here to view code image

```
0:000> !peb
PEB at 000000dd4c901000
   InheritedAddressSpace:    No
   ReadImageFileExecOptions: No
   BeingDebugged:            Yes
   ImageBaseAddress:         00007ff720b60000
   Ldr                       00007ffe855d23a0
   Ldr.Initialized:          Yes
   Ldr.InInitializationOrderModuleList: 0000022815d23d30 .
0000022815d24430
   Ldr.InLoadOrderModuleList:           0000022815d23ee0 .
0000022815d31240
   Ldr.InMemoryOrderModuleList:         0000022815d23ef0 .
0000022815d31250
                Base TimeStamp                     Module
        7ff720b60000 5789986a Jul 16 05:14:02 2016 C:\Windows\System32\
notepad.exe
        7ffe85480000 5825887f Nov 11 10:59:43 2016
C:\WINDOWS\SYSTEM32\
ntdll.dll
        7ffe84bd0000 57899a29 Jul 16 05:21:29 2016
C:\WINDOWS\System32\
KERNEL32.DLL
        7ffe823c0000 582588e6 Nov 11 11:01:26 2016
C:\WINDOWS\System32\
KERNELBASE.dll
...
```

2. The address shown on the `Ldr` line is a pointer to the `PEB_LDR_DATA` structure described earlier. Notice that WinDbg shows you the address of

the three lists, and dumps the initialization order list for you, displaying the full path, time stamp, and base address of each module.

3. You can also analyze each module entry on its own by going through the module list and then dumping the data at each address, formatted as a `LDR_DATA_TABLE_ENTRY` structure. Instead of doing this for each entry, however, WinDbg can do most of the work by using the `!list` extension and the following syntax:

Click here to view code image

```
!list -x "dt ntdll!_LDR_DATA_TABLE_ENTRY" @@C++(&@$peb->Ldr->InLoadOrderModuleList)
```

4. You should then see the entries for each module:

Click here to view code image

```
+0x000 InLoadOrderLinks : _LIST_ENTRY [ 0x00000228'15d23d10 - 0x00007ffe'855d23b0 ]
   +0x010 InMemoryOrderLinks : _LIST_ENTRY [ 0x00000228'15d23d20 - 0x00007ffe'855d23c0 ]
   +0x020 InInitializationOrderLinks : _LIST_ENTRY [ 0x00000000'00000000 - 0x00000000'00000000 ]
   +0x030 DllBase          : 0x00007ff7'20b60000 Void
   +0x038 EntryPoint       : 0x00007ff7'20b787d0 Void
   +0x040 SizeOfImage      : 0x41000
   +0x048 FullDllName      : _UNICODE_STRING "C:\Windows\System32\notepad.
exe"
   +0x058 BaseDllName      : _UNICODE_STRING "notepad.exe"
   +0x068 FlagGroup        : [4]  "???"
   +0x068 Flags            : 0xa2cc
```

Although this section covers the user-mode loader in Ntdll.dll, note that the kernel also employs its own loader for drivers and dependent DLLs, with a similar loader entry structure called `KLDR_DATA_TABLE_ENTRY` instead. Likewise, the kernel-mode loader has its own database of such entries, which is directly accessible through the `PsLoadedModuleList` global data variable. To dump the kernel's loaded module database, you can use a similar `!list` command as shown in the preceding experiment

by replacing the pointer at the end of the command with `nt!PsLoadedModuleList` and using the new structure/module name: `!list -x " dt nt!_kldr_data_table_entry" nt!PsLoadedModuleList`.

Looking at the list in this raw format gives you some extra insight into the loader's *internals*, such as the `Flags` field, which contains state information that `!peb` on its own would not show you. See **Table 3-10** for their meaning. Because both the kernel and user-mode loaders use this structure, the meaning of the flags is not always the same. In this table, we explicitly cover the user-mode flags only (some of which may exist in the kernel structure as well).

Flag	Meaning
Packaged Binary (0x1)	This module is part of a packaged application (it can only be set on the main module of an AppX package).
Marked for Removal (0x2)	This module will be unloaded as soon as all references (such as from an executing worker thread) are dropped.
Image DLL (0x4)	This module is an image DLL (and not a data DLL or executable).
Load Notifications Sent (0x8)	Registered DLL notification callouts were notified of this image already.
Telemetry Entry Processed (0x10)	Telemetry data has already been processed for this image.
Process Static Import (0x20)	This module is a static import of the main application binary.
In Legacy Lists (0x40)	This image entry is in the loader's doubly linked lists.
In Indexes (0x80)	This image entry is in the loader's red-black trees.
Shim DLL (0x100)	This image entry represents a DLL part of the shim engine/application compatibility database.
In Exception Table (0x200)	This module's .pdata exception handlers have been captured in the loader's inverted function table.
Load In Progress (0x800)	This module is currently being loaded.
Load Config Processed (0x1000)	This module's image load configuration directory has been found and processed.
Entry Processed (0x2000)	The loader has fully finished processing this module.
Protect Delay Load (0x4000)	Control Flow Guard features for this binary have requested the protection of the delay-load IAT. See chapter 7 for more information.
Process Attach Called (0x20000)	The DLL_PROCESS_ATTACH notification has already been sent to the DLL.
Process Attach Failed (0x40000)	The DllMain routine of the DLL has failed the DLL_PROCESS_ATTACH notification.
Don't Call for Threads (0x80000)	Do not send DLL_THREAD_ATTACH/DETACH notifications to this DLL. Can be set with `DisableThreadLibraryCalls`.
COR Deferred Validate (0x100000)	The Common Object Runtime (COR) will validate this .NET image at a later time.
COR Image (0x200000)	This module is a .NET application.
Don't Relocate (0x400000)	This image should not be relocated or randomized.
COR IL Only (0x800000)	This is a .NET intermediate-language (IL)-only library, which does not contain native assembly code.
Compat Database Processed (0x40000000)	The shim engine has processed this DLL.

TABLE 3-10 Loader data table entry flags

Import parsing

Now that we've explained the way the loader keeps track of all the modules loaded for a process, you can continue analyzing the startup initialization tasks performed by the loader. During this step, the loader will do the following:

1. Load each DLL referenced in the import table of the process's executable image.

2. Check whether the DLL has already been loaded by checking the module database. If it doesn't find it in the list, the loader opens the DLL and maps it into memory.

3. During the mapping operation, the loader first looks at the various paths where it should attempt to find this DLL, as well as whether this DLL is a *known DLL*, meaning that the system has already loaded it at startup and provided a global memory mapped file for accessing it. Certain deviations from the standard lookup algorithm can also occur, either through the use of a .local file (which forces the loader to use DLLs in the local path) or through a manifest file, which can specify a redirected DLL to use to guarantee a specific version.

4. After the DLL has been found on disk and mapped, the loader checks whether the kernel has loaded it somewhere else—this is called *relocation*. If the loader detects relocation, it parses the relocation information in the DLL and performs the operations required. If no relocation information is present, DLL loading fails.

5. The loader then creates a loader data table entry for this DLL and inserts it into the database.

6. After a DLL has been mapped, the process is repeated for this DLL to parse its import table and all its dependencies.

7. After each DLL is loaded, the loader parses the IAT to look for specific functions that are being imported. Usually this is done by name, but it can also be done by ordinal (an index number). For each name, the loader parses the export table of the imported DLL and tries to locate a match. If no match is found, the operation is aborted.

8. The import table of an image can also be bound. This means that at link time, the developers already assigned static addresses pointing to im-

ported functions in external DLLs. This removes the need to do the lookup for each name, but it assumes that the DLLs the application will use will always be located at the same address. Because Windows uses address space randomization (see **Chapter 5** for more information on ASLR), this is usually not the case for system applications and libraries.

9. The export table of an imported DLL can use a forwarder entry, meaning that the actual function is implemented in another DLL. This must essentially be treated like an import or dependency, so after parsing the export table, each DLL referenced by a forwarder is also loaded and the loader goes back to step 1.

After all imported DLLs (and their own dependencies, or imports) have been loaded, all the required imported functions have been looked up and found, and all forwarders also have been loaded and processed, the step is complete: All dependencies that were defined at compile time by the application and its various DLLs have now been fulfilled. During execution, delayed dependencies (called *delay load*), as well as run-time operations (such as calling `LoadLibrary`) can call into the loader and essentially repeat the same tasks. Note, however, that a failure in these steps will result in an error launching the application if they are done during process startup. For example, attempting to run an application that requires a function that isn't present in the current version of the operating system can result in a message similar to the one in **Figure 3-12**.

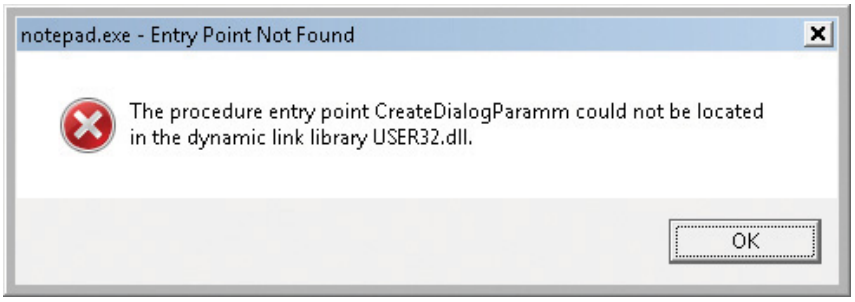


FIGURE 3-12 The dialog box shown when a required (imported) function is not present in a DLL.

Post-import process initialization

After the required dependencies have been loaded, several initialization tasks must be performed to fully finalize launching the application. In this phase, the loader will do the following:

1. These steps begin with the `LdrInitState` variable set to `2`, which means imports have loaded.

2. The initial debugger breakpoint will be hit when using a debugger such as WinDbg. This is where you had to type **g** to continue execution in earlier experiments.

3. Check if this is a Windows subsystem application, in which case the `BaseThreadInitThunk` function should've been captured in the early process initialization steps. At this point, it is called and checked for success. Similarly, the `TermsrvGetWindowsDirectoryW` function, which should have been captured earlier (if on a system which supports terminal services), is now called, which resets the System and Windows directories path.

4. Using the distributed graph, recurse through all dependencies and run the initializers for all of the images' static imports. This is the step that calls the `DllMain` routine for each DLL (allowing each DLL to perform its own initialization work, which might even include loading new DLLs at run time) as well as processes the TLS initializers of each DLL. This is one of the last steps in which loading an application can fail. If all the loaded DLLs do not return a successful return code after finishing their `DllMain` routines, the loader aborts starting the application.

5. If the image uses any TLS slots, call its TLS initializer.

6. Run the post-initialization shim engine callback if the module is being shimmed for application compatibility.

7. Run the associated subsystem DLL post-process initialization routine registered in the PEB. For Windows applications, this does Terminal Services–specific checks, for example.

8. At this point, write an ETW event indicating that the process has loaded successfully.

9. If there is a minimum stack commit, touch the thread stack to force an in-page of the committed pages.

10. Set `LdrInitState` to `3`, which means initialization done. Set the PEB's `ProcessInitializing` field back to `0`. Then, update the `LdrpProcessInitialized` variable.

SwitchBack

As each new version of Windows fixes bugs such as race conditions and incorrect parameter validation checks in existing API functions, an application-compatibility risk is created for each change, no matter how minor. Windows makes use of a technology called *SwitchBack*, implemented in the loader, which enables software developers to embed a GUID specific to the Windows version they are targeting in their executable's associated manifest.

For example, if a developer wants to take advantage of improvements added in Windows 10 to a given API, she would include the Windows 10 GUID in her manifest, while if a developer has a legacy application that depends on Windows 7–specific behavior, she would put the Windows 7 GUID in the manifest instead.

SwitchBack parses this information and correlates it with embedded information in SwitchBack-compatible DLLs (in the `.sb_data` image section) to decide which version of an affected API should be called by the module. Because SwitchBack works at the loaded-module level, it enables a process to have both legacy and current DLLs concurrently calling the same API, yet observing different results.

SwitchBack GUIDs

Windows currently defines GUIDs that represent compatibility settings for every version from Windows Vista:

- {e2011457-1546-43c5-a5fe-008deee3d3f0} for Windows Vista
- {35138b9a-5d96-4fbd-8e2d-a2440225f93a} for Windows 7
- {4a2f28e3-53b9-4441-ba9c-d69d4a4a6e38} for Windows 8
- {1f676c76-80e1-4239-95bb-83d0f6d0da78} for Windows 8.1
- {8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a} for Windows 10

These GUIDs must be present in the application's manifest file under the `<SupportedOS>` element in the ID attribute in a compatibility attribute entry. (If the application manifest does not contain a GUID, Windows Vista is chosen as the default compatibility mode.) Using Task Manager, you can enable an Operating System Context column in the Details tab,

which will show if any applications are running with a specific OS context (an empty value usually means they are operating in Windows 10 mode). **Figure 3-13** shows an example of a few such applications, which are operating in Windows Vista and Windows 7 modes, even on a Windows 10 system.

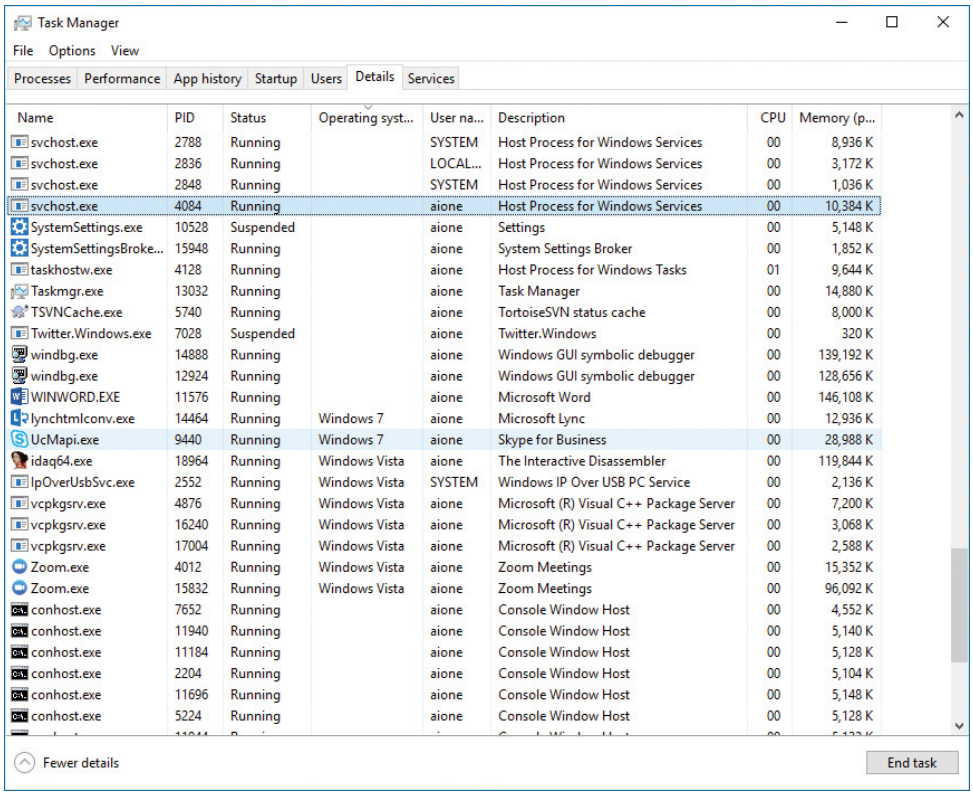


FIGURE 3-13 Some processes that run with compatibility modes.

Here is an example of a manifest entry that sets compatibility for Windows 10:

Click here to view code image

```
<compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">
  <application>
    <!-- Windows 10 -->
    <supportedOS Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}" />
  </application>
</compatibility>
```

SwitchBack compatibility modes

As a few examples of what SwitchBack can do, here's what running under the Windows 7 context affects:

- RPC components use the Windows thread pool instead of a private implementation.

- DirectDraw Lock cannot be acquired on the primary buffer.

- Blitting on the desktop is not allowed without a clipping window.

- A race condition in `GetOverlappedResult` is fixed.

- Calls to `CreateFile` are allowed to pass a “downgrade” flag to receive exclusive open to a file even when the caller does not have write privilege, which causes `NtCreateFile` not to receive the `FILE_DISALLOW_EXCLUSIVE` flag.

Running in Windows 10 mode, on the other hand, subtly affects how the Low Fragmentation Heap (LFH) behaves, by forcing LFH sub-segments to be fully committed and padding all allocations with a header block unless the Windows 10 GUID is present. Additionally, in Windows 10, using the Raise Exception on Invalid Handle Close mitigation (see **Chapter 7** for more information) will result in `CloseHandle` and `RegCloseKey` respecting the behavior. On the other hand, on previous operating systems, if the debugger is not attached, this behavior will be disabled before calling `NtClose`, and then re-enabled after the call.

As another example, the Spell Checking Facility will return NULL for languages which don’t have a spell checker, while it returns an “empty” spell checker on Windows 8.1. Similarly, the implementation of the function `IShellLink::Resolve` will return `E_INVALIDARG` when operating in Windows 8 compatibility mode when given a relative path, but will not contain this check in Windows 7 mode.

Furthermore, calls to `GetVersionEx` or the equivalent functions in NtDll such as `RtlVerifyVersion- Info` will return the maximum version number that corresponds to the SwitchBack Context GUID that was specified.

NOTE

These APIs have been deprecated, and calls to `GetVersionEx` will return 6.2 on all versions of Windows 8 and later if a higher SwitchBack GUID is not provided.

SwitchBack behavior

Whenever a Windows API is affected by changes that might break compatibility, the function's entry code calls the `SbSwitchProcedure` to invoke the SwitchBack logic. It passes along a pointer to the SwitchBack *module table*, which contains information about the SwitchBack mechanisms employed in the module. The table also contains a pointer to an array of entries for each SwitchBack point. This table contains a description of each branch-point that identifies it with a symbolic name and a comprehensive description, along with an associated mitigation tag. Typically, there will be several branch-points in a module, one for Windows Vista behavior, one for Windows 7 behavior, etc.

For each branch-point, the required SwitchBack context is given—it is this context that determines which of the two (or more) branches is taken at runtime. Finally, each of these descriptors contains a function pointer to the actual code that each branch should execute. If the application is running with the Windows 10 GUID, this will be part of its SwitchBack context, and the `SbSelectProcedure` API, upon parsing the module table, will perform a match operation. It finds the module entry descriptor for the context and proceeds to call the function pointer included in the descriptor.

SwitchBack uses ETW to trace the selection of given SwitchBack contexts and branch-points and feeds the data into the Windows AIT (Application Impact Telemetry) logger. This data can be periodically collected by Microsoft to determine the extent to which each compatibility entry is being used, identify the applications using it (a full stack trace is provided in the log), and notify third-party vendors.

As mentioned, the compatibility level of the application is stored in its manifest. At load time, the loader parses the manifest file, creates a context data structure, and caches it in the `pShimData` member of the PEB. This context data contains the associated compatibility GUIDs that this process is executing under and determines which version of the branch-points in the called APIs that employ SwitchBack will be executed.

API Sets

While SwitchBack uses API redirection for specific application-compatibility scenarios, there is a much more pervasive redirection mechanism used in Windows for all applications, called *API Sets*. Its purpose is to enable fine-grained categorization of Windows APIs into sub-DLLs instead of having large multi-purpose DLLs that span nearly thousands of APIs that might not be needed on all types of Windows systems today and in the future. This technology, developed mainly to support the refactoring of the bottom-most layers of the Windows architecture to separate it from higher layers, goes hand in hand with the breakdown of Kernel32.dll and Advapi32.dll (among others) into multiple, virtual DLL files.

For example, **Figure 3-14** shows a screenshot of Dependency Walker where Kernel32.dll, which is a core Windows library, imports from many other DLLs, beginning with API-MS-WIN. Each of these DLLs contains a small subset of the APIs that Kernel32 normally provides, but together they make up the entire API surface exposed by Kernel32.dll. The CORE-STRING library, for instance, provides only the Windows base string functions.

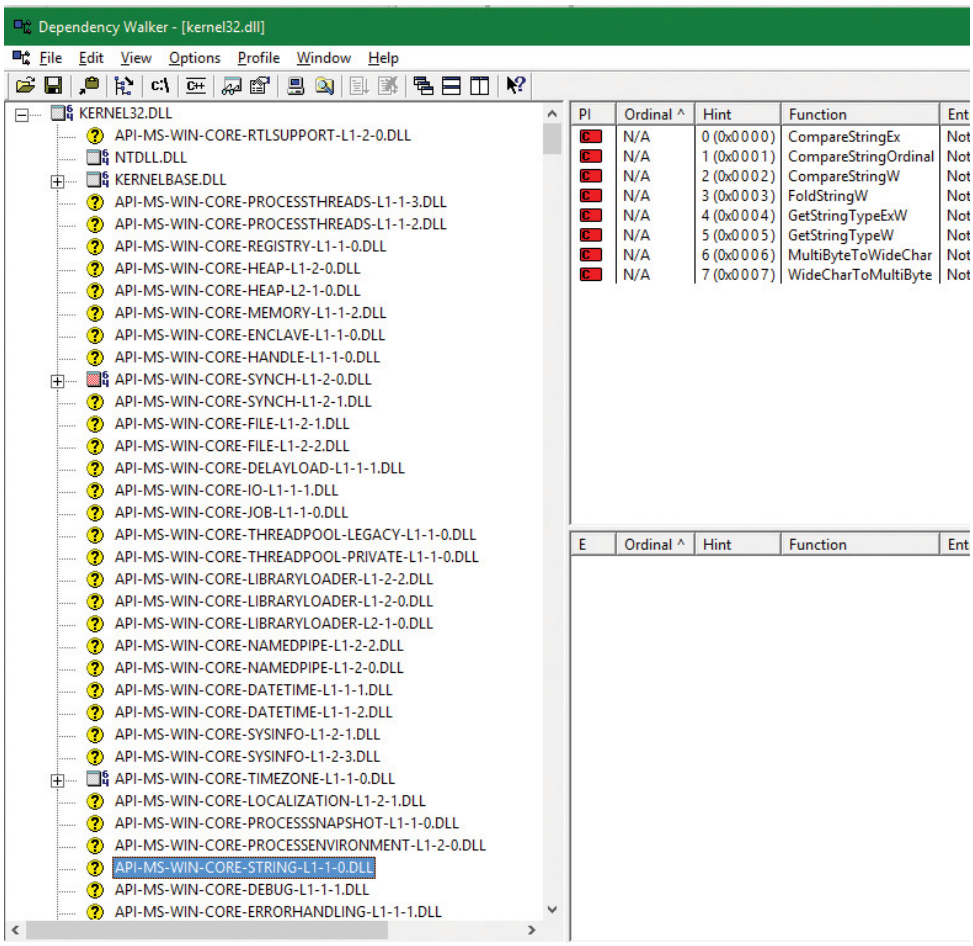


FIGURE 3-14 API sets for kernel32.dll.

In splitting functions across discrete files, two objectives are achieved. First, doing this allows future applications to link only with the API libraries that provide the functionality that they need. Second, if Microsoft were to create a version of Windows that did not support, for example, localization (say, a non-user-facing, English-only embedded system), it would be possible to simply remove the sub-DLL and modify the API Set schema. This would result in a smaller Kernel32 binary, and any applications that ran without requiring localization would still run.

With this technology, a "base" Windows system called *MinWin* is defined (and, at the source level, built), with a minimum set of services that includes the kernel, core drivers (including file systems, basic system processes such as CSRSS and the Service Control Manager, and a handful of Windows services). Windows Embedded, with its Platform Builder, provides what might seem to be a similar technology, as system builders are able to remove select "Windows components," such as the shell, or the network stack. However, removing components from Windows leaves *dangling dependencies*—code paths that, if exercised, would fail because they depend on the removed components. MinWin's dependencies, on the other hand, are entirely self-contained.

When the process manager initializes, it calls the `PspInitializeApiSetMap` function, which is responsible for creating a section object of the API Set redirection table, which is stored in %SystemRoot%\System32\ApiSetSchema.dll. The DLL contains no executable code, but it has a section called `.apiset` that contains API Set mapping data that maps virtual API Set DLLs to logical DLLs that implement the APIs. Whenever a new process starts, the process manager maps the section object into the process's address space and sets the `ApiSetMap` field in the process's PEB to point to the base address where the section object was mapped.

In turn, the loader's `LdrpApplyFileNameRedirection` function, which is normally responsible for the .local and SxS/Fusion manifest redirection that was mentioned earlier, also checks for API Set redirection data whenever a new import library that has a name starting with *API-* loads (either dynamically or statically). The API Set table is organized by library with each entry describing in which logical DLL the function can be found, and that DLL is what gets loaded. Although the schema data is a binary format, you can dump its strings with the Sysinternals Strings tool to see which DLLs are currently defined:

Click here to view code image

```
C:\Windows\System32>strings apisetschema.dll
...
api-ms-onecoreuap-print-render-l1-1-0
printrenderapihost.dllapi-ms-onecoreuap-settingsync-status-l1-1-0
settingsynccore.dll
api-ms-win-appmodel-identity-l1-2-0
kernel.appcore.dllapi-ms-win-appmodel-runtime-internal-l1-1-3
api-ms-win-appmodel-runtime-l1-1-2
api-ms-win-appmodel-state-l1-1-2
api-ms-win-appmodel-state-l1-2-0
api-ms-win-appmodel-unlock-l1-1-0
api-ms-win-base-bootconfig-l1-1-0
advapi32.dllapi-ms-win-base-util-l1-1-0
api-ms-win-composition-redirection-l1-1-0
...
api-ms-win-core-com-midlproxystub-l1-1-0
api-ms-win-core-com-private-l1-1-1
api-ms-win-core-comm-l1-1-0
api-ms-win-core-console-ansi-l2-1-0
api-ms-win-core-console-l1-1-0
api-ms-win-core-console-l2-1-0
api-ms-win-core-crt-l1-1-0
api-ms-win-core-crt-l2-1-0
api-ms-win-core-datetime-l1-1-2
api-ms-win-core-debug-l1-1-2
api-ms-win-core-debug-minidump-l1-1-0
...
api-ms-win-core-firmware-l1-1-0
api-ms-win-core-guard-l1-1-0
api-ms-win-core-handle-l1-1-0
api-ms-win-core-heap-l1-1-0
api-ms-win-core-heap-l1-2-0
api-ms-win-core-heap-l2-1-0
api-ms-win-core-heap-obsolete-l1-1-0
api-ms-win-core-interlocked-l1-1-1
api-ms-win-core-interlocked-l1-2-0
api-ms-win-core-io-l1-1-1
api-ms-win-core-job-l1-1-0
...
```

Jobs

A *job* is a nameable, securable, shareable kernel object that allows control of one or more processes as a group. A job object's basic function is to allow groups of processes to be managed and manipulated as a unit. A process can be a member of any number of jobs, although the typical case is just one. A process's association with a job object can't be broken, and all processes created by the process and its descendants are associated with the same job object (unless child processes are created with the `CREATE_BREAKAWAY_FROM_JOB` flag and the job itself has not restricted it). The job object also records basic accounting information for all processes associated with the job and for all processes that were associated with the job but have since terminated.

Jobs can also be associated with an I/O completion port object, which other threads might be waiting for, with the Windows `GetQueuedCompletionStatus` function or by using the Thread Pool API (the native function `TpAllocJobNotification`). This allows interested parties (typically the job creator) to monitor for limit violations and events that could affect the job's security, such as a new process being created or a process abnormally exiting.

Jobs play a significant role in a number of system mechanisms, enumerated here:

- They manage modern apps (UWP processes), as discussed in more detail in Chapter 9 in Part 2. In fact, every modern app is running under a job. You can verify this with Process Explorer, as described in the "**Viewing the job object**" experiment later in this chapter.

- They are used to implement Windows Container support, through a mechanism called *server silo*, covered later in this section.

- They are the primary way through which the Desktop Activity Moderator (DAM) manages throttling, timer virtualization, timer freezing, and other idle-inducing behaviors for Win32 applications and services. The DAM is described in Chapter 8 in Part 2.

- They allow the definition and management of scheduling groups for dynamic fair-share scheduling (DFSS), which is described in **Chapter 4**.

- They allow for the specification of a custom memory partition, which enables usage of the Memory Partitioning API described in **Chapter 5**.

■ They serve as a key enabler for features such as Run As (Secondary Logon), Application Boxing, and Program Compatibility Assistant.

■ They provide part of the security sandbox for applications such as Google Chrome and Microsoft Office Document Converter, as well as mitigation from denial-of-service (DoS) attacks through Windows Management Instrumentation (WMI) requests.

Job limits

The following are some of the CPU-, memory-, and I/O-related limits you can specify for a job:

■ **Maximum number of active processes** This limits the number of concurrently existing processes in the job. If this limit is reached, new processes that should be assigned to the job are blocked from creation.

■ **Job-wide user-mode CPU time limit** This limits the maximum amount of user-mode CPU time that the processes in the job can consume (including processes that have run and exited). Once this limit is reached, by default all the processes in the job are terminated with an error code and no new processes can be created in the job (unless the limit is reset). The job object is signaled, so any threads waiting for the job will be released. You can change this default behavior with a call to `SetInformationJobObject` to set the `EndOfJobTimeAction` member of the `JOBOBJECT_END_OF_JOB_TIME_INFORMATION` structure passed with the `JobObjectEndOfJobTimeInformation` information class and request a notification to be sent through the job's completion port instead.

■ **Per-process user-mode CPU time limit** This allows each process in the job to accumulate only a fixed maximum amount of user-mode CPU time. When the maximum is reached, the process terminates (with no chance to clean up).

■ **Job processor affinity** This sets the processor affinity mask for each process in the job. (Individual threads can alter their affinity to any subset of the job affinity, but processes can't alter their process affinity setting.)

■ **Job group affinity** This sets a list of groups to which the processes in the job can be assigned. Any affinity changes are then subject to the group selection imposed by the limit. This is treated as a group-aware

version of the job processor affinity limit (legacy), and prevents that limit from being used.

■ **Job process priority class** This sets the priority class for each process in the job. Threads can't increase their priority relative to the class (as they normally can). Attempts to increase thread priority are ignored. (No error is returned on calls to `SetThreadPriority`, but the increase doesn't occur.)

■ **Default working set minimum and maximum** This defines the specified working set minimum and maximum for each process in the job. (This setting isn't job-wide. Each process has its own working set with the same minimum and maximum values.)

■ **Process and job committed virtual memory limit** This defines the maximum amount of virtual address space that can be committed by either a single process or the entire job.

■ **CPU rate control** This defines the maximum amount of CPU time that the job is allowed to use before it will experience forced throttling. This is used as part of the scheduling group support described in **Chapter 4**.

■ **Network bandwidth rate control** This defines the maximum outgoing bandwidth for the entire job before throttling takes effect. It also enables setting a differentiated services code point (DSCP) tag for QoS purposes for each network packet sent by the job. This can only be set for one job in a hierarchy, and affects the job and any child jobs.

■ **Disk I/O bandwidth rate control** This is the same as network bandwidth rate control, but is applied to disk I/O instead, and can control either bandwidth itself or the number of I/O operations per second (IOPS). It can be set either for a particular volume or for all volumes on the system.

For many of these limits, the job owner can set specific thresholds, at which point a notification will be sent (or, if no notification is registered, the job will simply be killed). Additionally, rate controls allow for tolerance ranges and tolerance intervals—for example, allowing a process to go beyond 20 percent of its network bandwidth limit for up to 10 seconds every 5 minutes. These notifications are done by queuing an appropriate message to the I/O completion port for the job. (See the Windows SDK documentation for the details.)

Finally, you can place user-interface limits on processes in a job. Such limits include restricting processes from opening handles to windows owned by threads outside the job, reading and/or writing to the clipboard, and changing the many user-interface system parameters via the Windows `SystemParametersInfo` function. These user-interface limits are managed by the Windows subsystem GDI/USER driver, Win32k.sys, and are enforced through one of the special callouts that it registers with the process manager, the job callout. You can grant access for all processes in a job to specific user handles (for example, window handle) by calling the `UserHandleGrantAccess` function; this can only be called by a process that is not part of the job in question (naturally).

Working with a job

A job object is created using the `CreateJobObject` API. The job is initially created empty of any process. To add a process to a job, call the `AssignProcessToJobObject`, which can be called multiple times to add processes to the job or even to add the same process to multiple jobs. This last option creates a nested job, described in the next section. Another way to add a process to a job is to manually specify a handle to the job object by using the `PS_CP_JOB_LIST` process-creation attribute described earlier in this chapter. One or more handles to job objects can be specified, which will all be joined.

The most interesting API for jobs is `SetInformationJobObject`, which allows the setting of the various limits and settings mentioned in the previous section, and contains internal information classes used by mechanisms such as Containers (Silo), the DAM, or Windows UWP applications. These values can be read back with `QueryInformationJobObject`, which can provide interested parties with the limits set on a job. It's also necessary to call in case limit notifications have been set (as described in the previous section) in order for the caller to know precisely which limits were violated. Another sometimes-useful function is `TerminateJobObject`, which terminates all processes in the job (as if `TerminateProcess` were called on each process).

Nested jobs

Until Windows 7 and Windows Server 2008 R2, a process could only be associated with a single job, which made jobs less useful than they could be, as in some cases an application could not know in advance whether a process it needed to manage happened to be in a job or not. Starting with Windows 8 and Windows Server 2012, a process can be associated with multiple jobs, effectively creating a job hierarchy.

A child job holds a subset of processes of its parent job. Once a process is added to more than one job, the system tries to form a hierarchy, if possible. A current restriction is that jobs cannot form a hierarchy if any of them sets any UI limits (`SetInformationJobObject` with `JobObjectBasicUIRestrictions` argument).

Job limits for a child job cannot be more permissive than its parent, but they can be more restrictive. For example, if a parent job sets a memory limit of 100 MB for the job, any child job cannot set a higher memory limit (such requests simply fail). A child job can, however, set a more restrictive limit for its processes (and any child jobs it has), such as 80 MB. Any notifications that target the I/O completion port of a job will be sent to the job and all its ancestors. (The job itself does not have to have an I/O completion port for the notification to be sent to ancestor jobs.)

Resource accounting for a parent job includes the aggregated resources used by its direct managed processes and all processes in child jobs. When a job is terminated (`TerminateJobObject`), all processes in the job and in child jobs are terminated, starting with the child jobs at the bottom of the hierarchy. **Figure 3-15** shows four processes managed by a job hierarchy.

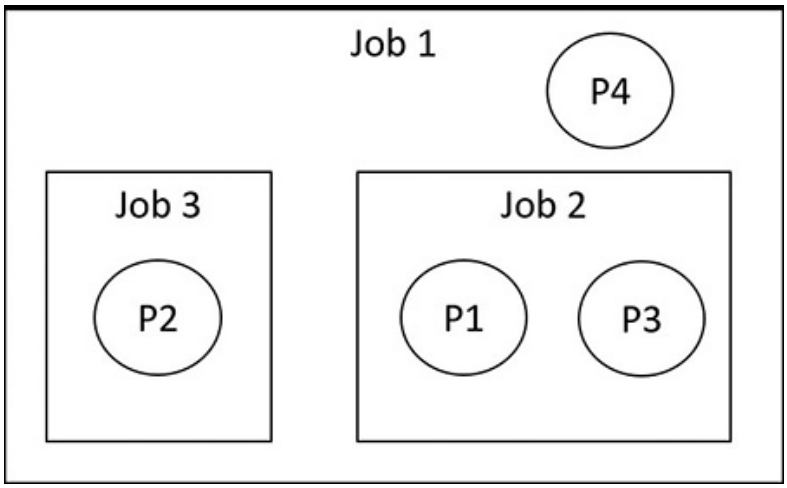


FIGURE 3-15 A job hierarchy.

To create this hierarchy, processes should be added to jobs from the root job. Here are a set of steps to create this hierarchy:

1. Add process P1 to job 1.

2. Add process P1 to job 2. This creates the first nesting.

3. Add process P2 to job 1.

4. Add process P2 to job 3. This creates the second nesting.

5. Add process P3 to job 2.

6. Add process P4 to job 1.

EXPERIMENT: VIEWING THE JOB OBJECT

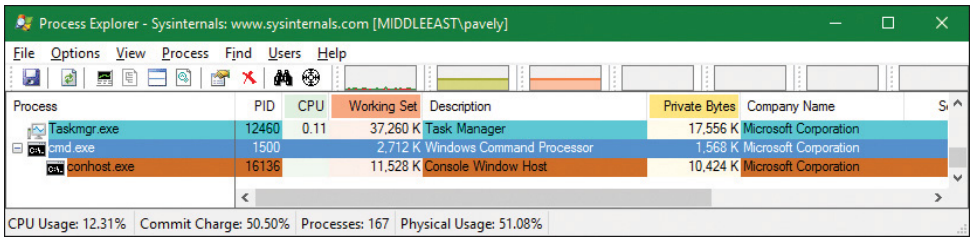
You can view named job objects with the Performance Monitor tool. (Look for the Job Object and Job Object Details categories.) You can view unnamed jobs with the kernel debugger `!job` or `dt nt!_ejob` commands.

To see whether a process is associated with a job, you can use the kernel debugger `!process` command or Process Explorer. Follow these steps to create and view an unnamed job object:

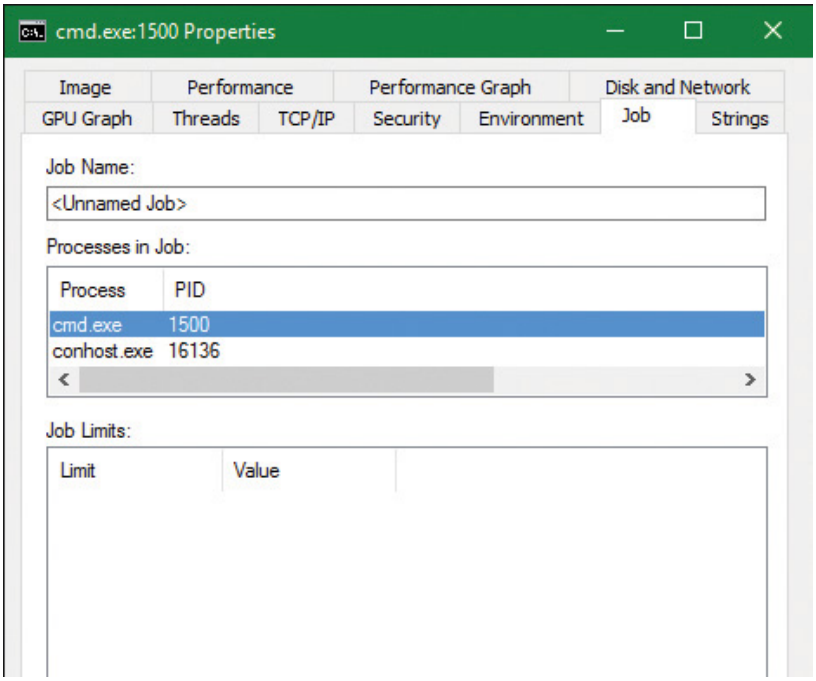
1. From the command prompt, use the `runas` command to create a process running the command prompt (Cmd.exe). For example, type **runas /user:<domain>\< username> cmd**.

2. You'll be prompted for your password. Enter your password, and a Command Prompt window will appear. The Windows service that executes `runas` commands creates an unnamed job to contain all processes (so that it can terminate these processes at logoff time).

3. Run Process Explorer, open the **Options** menu, choose **Configure Colors**, and check the **Jobs** entry. Notice that the Cmd.exe process and its child ConHost.exe process are highlighted as part of a job, as shown here:



4. Double click the **Cmd.exe** or **ConHost.Exe** process to open its properties dialog box. Then click the **Job** tab to see information about the job this process is part of:



5. From the command prompt, run Notepad.exe.

6. Open Notepad's process and look at the Job tab. Notepad is running under the same job. This is because cmd.exe does not use the `CREATE_BREAKAWAY_FROM_JOB` creation flag. In the case of nested jobs, the Job tab shows the processes in the direct job this process belongs to and all processes in child jobs.

7. Run the kernel debugger on the live system and type the **!process** command to find the notepad.exe and show its basic info:

Click here to view code image

```
lkd> !process 0 1 notepad.exe
PROCESS ffffe001eacf2080
    SessionId: 1  Cid: 3078    Peb: 7f4113b000  ParentCid: 05dc
    DirBase: 4878b3000  ObjectTable: ffffc0015b89fd80  HandleCount: 188.
    Image: notepad.exe
    ...
    BasePriority               8
    CommitCharge               671
    Job                        ffffe00189aec460
```

8. Note the `Job` pointer, which is non-zero. To get a summary of the job, type the **!job** debugger command:

Click here to view code image

```
lkd> !job ffffe00189aec460
Job at ffffe00189aec460
  Basic Accounting Information
    TotalUserTime:             0x0
    TotalKernelTime:           0x0
    TotalCycleTime:            0x0
    ThisPeriodTotalUserTime:   0x0
    ThisPeriodTotalKernelTime: 0x0
    TotalPageFaultCount:       0x0
    TotalProcesses:            0x3
    ActiveProcesses:           0x3
    FreezeCount:               0
    BackgroundCount:           0
    TotalTerminatedProcesses:  0x0
    PeakJobMemoryUsed:         0x10db
    PeakProcessMemoryUsed:     0xa56
  Job Flags
  Limit Information (LimitFlags: 0x0)
  Limit Information (EffectiveLimitFlags: 0x0)
```

9. Notice the `ActiveProcesses` member set to `3` (cmd.exe, conhost.exe, and notepad.exe). You can use flag `2` after the `!job` command to see a list of the processes that are part of the job:

Click here to view code image

```
lkd> !job ffffe00189aec460 2
...
Processes assigned to this job:
    PROCESS ffff8188d84dd780
        SessionId: 1  Cid: 5720    Peb: 43bedb6000  ParentCid: 13cc
        DirBase: 707466000  ObjectTable: fffffbe0dc4e3a040  HandleCount:
<Data Not Accessible>
        Image: cmd.exe

    PROCESS ffff8188ea077540
        SessionId: 1  Cid: 30ec    Peb: dd7f17c000  ParentCid: 5720
        DirBase: 75a183000  ObjectTable: fffffbe0dafb79040  HandleCount:
<Data Not Accessible>
        Image: conhost.exe
```

PROCESS ffffe001eacf2080
SessionId: 1 Cid: 3078 Peb: 7f4113b000 ParentCid: 05dc
DirBase: 4878b3000 ObjectTable: ffffc0015b89fd80 HandleCount: 188.
Image: notepad.exe

10. You can also use the `dt` command to display the job object and see the additional fields shown about the job, such as its member level and its relations to other jobs in case of nesting (parent job, siblings, and root job):

Click here to view code image

```
lkd> dt nt!_ejob ffffe00189aec460
   +0x000 Event          : _KEVENT
   +0x018 JobLinks       : _LIST_ENTRY [ 0xffffe001'8d93e548 -
0xffffe001'df30f8d8 ]
   +0x028 ProcessListHead  : _LIST_ENTRY [ 0xffffe001'8c4924f0 -
0xffffe001'eacf24f0 ]
   +0x038 JobLock        : _ERESOURCE
   +0x0a0 TotalUserTime    : _LARGE_INTEGER 0x0
   +0x0a8 TotalKernelTime  : _LARGE_INTEGER 0x2625a
   +0x0b0 TotalCycleTime   : _LARGE_INTEGER 0xc9e03d
   ...
   +0x0d4 TotalProcesses   : 4
   +0x0d8 ActiveProcesses  : 3
   +0x0dc TotalTerminatedProcesses : 0
...
   +0x428 ParentJob      : (null)
   +0x430 RootJob        : 0xffffe001'89aec460 _EJOB
...
   +0x518 EnergyValues     : 0xffffe001'89aec988
_PROCESS_ENERGY_VALUES
   +0x520 SharedCommitCharge : 0x5e8
```

Windows containers (server silos)

The rise of cheap, ubiquitous cloud computing has led to another major Internet revolution, in which building online services and/or back-end servers for mobile applications is as easy as clicking a button on one of the many cloud providers. But as competition among cloud providers has increased, and as the need to migrate from one to another, or even from a

cloud provider to a datacenter, or from a datacenter to a high-end personal server, has grown, it has become increasingly important to have portable back ends, which can be deployed and moved around as needed without the costs associated with running them in a virtual machine. It is to satisfy this need that technologies such as Docker were created. These technologies essentially allow the deployment of an "application in a box" from one Linux distribution to another without worrying about the complicated deployment of a local installation or the resource consumption of a virtual machine. Originally a Linux-only technology, Microsoft has helped bring Docker to Windows 10 as part of the Anniversary Update. It can work in two modes:

- By deploying an application in a heavyweight, but fully isolated, Hyper-V container, which is supported on both client and server scenarios

- By deploying an application in a lightweight, OS-isolated, server silo container, which is currently supported only in server scenarios due to licensing reasons

This latter technology, which we will investigate in this section, has resulted in deep changes in the operating system to support this capability. Note that, as mentioned, the ability for client systems to create server silo containers exists, but is currently disabled. Unlike a Hyper-V container, which leverages a true virtualized environment, a server silo container provides a second "instance" of all user-mode components while running on top of the same kernel and drivers. At the cost of some security, this provides a much more lightweight container environment.

Job objects and silos

The ability to create a silo is associated with a number of undocumented subclasses as part of the `SetJobObjectInformation` API. In other words, a silo is essentially a super-job, with additional rules and capabilities beyond those we've seen so far. In fact, a job object can be used for the isolation and resource management capabilities we've looked at as well as used to create a silo. Such jobs are called *hybrid jobs* by the system.

In practice, job objects can actually host two types of silos: application silos (which are currently used to implement the Desktop Bridge are not covered in this section, and are left for Chapter 9 in Part 2) and server silos, which are the ones used for Docker container support.

Silo isolation

The first element that defines a server silo is the existence of a custom object manager root directory object (\). (The object manager is discussed in Chapter 8 in Part 2.) Even though we have not yet learned about this mechanism, suffice it to say that all application-visible named objects (such as files, registry keys, events, mutexes, RPC ports, and more) are hosted in a root namespace, which allows applications to create, locate, and share these objects among themselves.

The ability for a server silo to have its own root means that all access to any named object can be controlled. This is done in one of three ways:

- By creating a new copy of an existing object to provide an alternate access to it from within the silo

- By creating a symbolic link to an existing object to provide direct access to it

- By creating a brand-new object that only exists within the silo, such as the ones a containerized application would use

This initial ability is then combined with the Virtual Machine Compute (Vmcompute) service (used by Docker), which interacts with additional components to provide a full isolation layer:

- **A base Windows image (WIM) file called base OS** This provides a separate copy of the operating system. At this time, Microsoft provides a Server Core image as well as a Nano Server image.

- **The Ntdll.dll library of the host OS** This overrides the one in the base OS image. This is due to the fact that, as mentioned, server silos leverage the same host kernel and drivers, and because Ntdll.dll handles system calls, it is the one user-mode component that must be reused from the host OS.

- **A sandbox virtual file system provided by the Wcifs.sys filter driver** This allows temporary changes to be made to the file system by the container without affecting the underlying NTFS drive, and which can be wiped once the container is shut down.

- **A sandbox virtual registry provided by the VReg kernel component** This allows for the provision of a temporary set of registry hives (as well

as another layer of namespace isolation, as the object manager root namespace only isolates the root of the registry, not the registry hives themselves).

■ **The Session Manager (Smss.exe)** This is now used to create additional service sessions or console sessions, which is a new capability required by the container support. This extends Smss to handle not only additional user sessions, but also sessions needed for each container launched.

The architecture of such containers with the preceding components is shown in **Figure 3-16**.

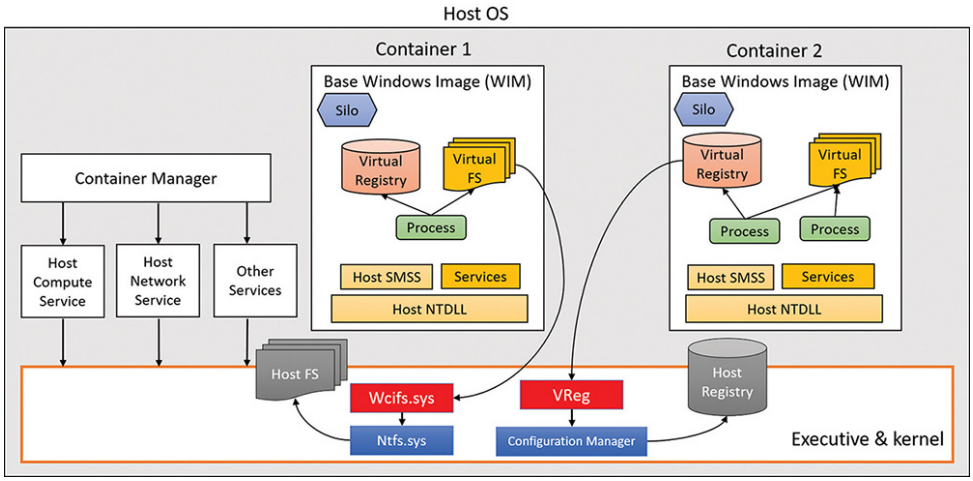


FIGURE 3-16 Containers architecture.

Silo isolation boundaries

The aforementioned components provide the user-mode isolation environment. However, as the host Ntdll.dll component is used, which talks to the host kernel and drivers, it is important to create additional isolation boundaries, which the kernel provides to differentiate one silo from another. As such, each server silo will contain its own isolated:

■ **Micro shared user data (`SILO_USER_SHARED_DATA` in the symbols)** This contains the custom system path, session ID, foreground PID, and product type/suite. These are elements of the original `KUSER_SHARED_DATA` that cannot come from the host, as they reference information relevant to the host OS image instead of the base OS image, which must be used instead. Various components and APIs were modified to read the silo shared data instead of the user shared data when they look up such data. Note that the original `KUSER_SHARED_DATA` remains at its usual address with its original view of the host details, so this is one way that host state "leaks" inside container state.

■ **Object directory root namespace** This has its own \SystemRoot symlink, \Device directory (which is how all user-mode components access device drivers indirectly), device map and DOS device mappings (which is how user-mode applications access network mapped drivers, for example), \Sessions directory, and more.

■ **API Set mapping** This is based on the API Set schema of the base OS WIM, and not the one stored on the host OS file system. As you've seen, the loader uses API Set mappings to determine which DLL, if any, implements a certain function. This can be different from one SKU to another, and applications must see the base OS SKU, not the host's.

■ **Logon session** This is associated with the `SYSTEM` and `Anonymous` local unique ID (LUID), plus the LUID of a virtual service account describing the user in the silo. This essentially represents the token of the services and application that will be running inside the container service session created by Smss. For more information on LUIDs and logon sessions, see **Chapter 7**.

■ **ETW tracing and logger contexts** These are for isolating ETW operations to the silo and not exposing or leaking states between the containers and/or the host OS itself. (See Chapter 9 in Part 2 for more on ETW.)

Silo contexts

While these are the isolation boundaries provided by the core host OS kernel itself, other components inside the kernel, as well as drivers (including third party), can add contextual data to silos by using the `PsCreateSiloContext` API to set custom data associated with a silo or by associating an existing object with a silo. Each such silo context will utilize a silo slot index that will be inserted in all running, and future, server silos, storing a pointer to the context. The system provides 32 built-in system-wide storage slot indexes, plus 256 expansion slots, providing lots of extensibility options.

As each server silo is created, it receives its own silo-local storage (SLS) array, much like a thread has thread-local storage (TLS). Within this array, the different entries will correspond to slot indices that have been allocated to store silo contexts. Each silo will have a different pointer at the same slot index, but will always store the same context at that index. (For example, driver "Foo" will own index 5 in all silos, and can use it to store a different pointer/context in each silo.) In some cases, built-in kernel components, such as the object manager, security reference monitor

(SRM), and Configuration Manager use some of these slots, while other slots are used by inbox drivers (such as the Ancillary Function Driver for Winsock, Afd.sys).

Just like when dealing with the server silo shared user data, various components and APIs have been updated to access data by getting it from the relevant silo context instead of what used to be a global kernel variable. As an example, because each container will now host its own Lsass.exe process, and since the kernel's SRM needs to own a handle to the Lsass.exe process (see **Chapter 7** for more information on Lsass and the SRM), this can no longer be a singleton stored in a global variable. As such, the handle is now accessed by the SRM through querying the silo context of the active server silo, and getting the variable from the data structure that is returned.

This leads to an interesting question: What happens with the Lsass.exe that is running on the host OS itself? How will the SRM access the handle, as there's no server silo for this set of processes and session (that is, session 0 itself)? To solve this conundrum, the kernel now implements a root host silo. In other words, the host itself is presumed to be part of a silo as well! This isn't a silo in the true sense of the word, but rather a clever trick to make querying silo contexts for the current silo work, even when there is no current silo. This is implemented by storing a global kernel variable called `PspHostSilo-Globals`, which has its own Slot Local Storage Array, as well as other silo contexts used by built-in kernel components. When various silo APIs are called with a `NULL` pointer, this `"NULL"` is instead treated as "no silo—i.e., use the host silo."

EXPERIMENT: DUMPING SRM SILO CONTEXT FOR THE HOST SILO

As shown, even though your Windows 10 system may not be hosting any server silos, especially if it is a client system, a host silo still exists, which contains the silo-aware isolated contexts used by the kernel. The Windows Debugger has an extension, `!silo`, which can be used with the `-g Host` parameters as follows: `!silo -g Host`. You should see output similar to the one below:

Click here to view code image

```
lkd> !silo -g Host
Server silo globals fffff801b73bc580:
                Default Error Port: ffffb30f25b48080
                ServiceSessionId  : 0
                Root Directory    : 00007fff00000000 ''
                State             : Running
```

In your output, the pointer to the silo globals should be hyperlinked, and clicking it will result in the following command execution and output:

Click here to view code image

```
lkd> dx -r1 (*((nt!_ESERVERSILO_GLOBALS *)0xfffff801b73bc580))
(*((nt!_ESERVERSILO_GLOBALS *)0xfffff801b73bc580))                [Type: _
ESERVERSILO_GLOBALS]
    [+0x000] ObSiloState      [Type: _OBP_SILODRIVERSTATE]
    [+0x2e0] SeSiloState      [Type: _SEP_SILOSTATE]
    [+0x310] SeRmSiloState    [Type: _SEP_RM_LSA_CONNECTION_STATE]
    [+0x360] CmSiloState      : 0xffffc308870931b0 [Type:
_CMP_SILO_CONTEXT *]
    [+0x368] EtwSiloState     : 0xffffb30f236c4000 [Type:
_ETW_SILODRIVERSTATE *]
...
```

Now click the `SeRmSiloState` field, which will expand to show you, among other things, a pointer to the Lsass.exe process:

Click here to view code image

```
lkd> dx -r1 ((ntkrnlmp!_SEP_RM_LSA_CONNECTION_STATE
*)0xfffff801b73bc890)
((ntkrnlmp!_SEP_RM_LSA_CONNECTION_STATE
*)0xfffff801b73bc890)      :
```

0xfffff801b73bc890 [Type: _SEP_RM_LSA_CONNECTION_STATE *]
[+0x000] LsaProcessHandle : 0xffffffff80000870 [Type: void *]
[+0x008] LsaCommandPortHandle : 0xffffffff8000087c [Type: void *]
[+0x010] SepRmThreadHandle : 0x0 [Type: void *]
[+0x018] RmCommandPortHandle : 0xffffffff80000874 [Type: void *]

Silo monitors

If kernel drivers have the capability to add their own silo contexts, how do they first know what silos are executing, and what new silos are created as containers are launched? The answer lies in the silo monitor facility, which provides a set of APIs to receive notifications whenever a server silo is created and/or terminated (`PsRegisterSiloMonitor`, `PsStartSiloMonitor`, `PsUnregisterSiloMonitor`), as well as notifications for any already-existing silos. Then, each silo monitor can retrieve its own slot index by calling `PsGetSiloMonitorContextSlot`, which it can then use with the `PsInsertSiloContext`, `PsReplaceSiloContext`, and `PsRemoveSiloContext` functions as needed. Additional slots can be allocated with `PsAllocSiloContextSlot`, but this would be needed only if a component would wish to store two contexts for some reason. Additionally, drivers can also use the `PsInsertPermanentSiloContext` or `PsMakeSiloContextPermanent` APIs to use "permanent" silo contexts, which are not reference counted and are not tied to the lifetime of the server silo or the number of silo context getters. Once inserted, such silo contexts can be retrieved with `PsGetSiloContext` and/or `PsGetPermanentSiloContext`.

EXPERIMENT: SILO MONITORS AND CONTEXTS

To understand how silo monitors are used, and how they store silo contexts, let's take a look at the Ancillary Function Driver for Winsock (Afd.sys) and its monitor. First, let's dump the data structure that represents the monitor. Unfortunately, it is not in the symbol files, so we must do this as raw data.

Click here to view code image

```
lkd> dps poi(afd!AfdPodMonitor)
ffffe387'a79fc120  ffffe387'a7d760c0
ffffe387'a79fc128  ffffe387'a7b54b60
ffffe387'a79fc130  00000009'00000101
ffffe387'a79fc138  fffff807'be4b5b10 afd!AfdPodSiloCreateCallback
ffffe387'a79fc140  fffff807'be4bee40 afd!AfdPodSiloTerminateCallback
```

Now get the slot (9 in this example) from the host silo. Silos store their SLS in a field called `Storage`, which contains an array of data structures (slot entries), each storing a pointer, and some flags. We are multiplying the index by 2 to get the offset of the right slot entry, then accessing the second field (+1) to get the pointer to the context pointer:

Click here to view code image

```
lkd> r? @$t0 =
(nt!_ESERVERSILO_GLOBALS*)@@masm(nt!PspHostSiloGlobals)
lkd> ?? ((void***)@$t0->Storage)[9 * 2 + 1]
void ** 0xffff988f'ab815941
```

Note that the permanent flag (0x2) is ORed into the pointer, mask it out, and then use the `!object` extension to confirm that this is truly a silo context.

Click here to view code image

```
lkd> !object (0xffff988f'ab815941 & -2)
Object: ffff988fab815940  Type: (ffff988faaac9f20) PsSiloContextNonPaged
```

Creation of a server silo

When a server silo is created, a job object is first used, because as mentioned, silos are a feature of job objects. This is done through the standard

`CreateJobObject` API, which was modified as part of the Anniversary Update to now have an associated job ID, or JID. The JID comes from the same pool of numbers as the process and thread ID (PID and TID), which is the client ID (CID) table. As such, a JID is unique among not only other jobs, but also other processes and threads. Additionally, a container GUID is automatically created.
Next, the `SetInformationJobObject` API is used, with the create silo information class. This results in the `Silo` flag being set inside of the EJOB executive object that represents the job, as well as the allocation of the SLS slot array we saw earlier in the `Storage` member of EJOB. At this point, we have an *application silo*.

After this, the root object directory namespace is created with another information class and call to `SetInformationJobObject`. This new class requires the trusted computing base (TCB) privilege. As silos are normally created only by the Vmcompute service, this is to ensure that virtual object namespaces are not used maliciously to confuse applications and potentially break them. When this namespace is created, the object manager creates or opens a new Silos directory under the real host root (\) and appends the JID to create a new virtual root (e.g., \Silos\148\). It then creates the `Kernel-Objects`, `ObjectTypes`, `GLOBALROOT`, and `DosDevices` objects. The root is then stored as a silo context with whatever slot index is in `PsObjectDirectorySiloContextSlot`, which was allocated by the object manager at boot.

The next step is to convert this silo into a server silo, which is done with yet another call to `Set-InformationJobObject` and another information class. The `PspConvertSiloToServerSilo` function in the kernel now runs, which initializes the `ESERVERSILO_GLOBALS` structure we saw earlier as part of the experiment dumping the `PspHostSiloGlobals` with the `!silo` command. This initializes the silo shared user data, API Set mapping, SystemRoot, and the various silo contexts, such as the one used by the SRM to identify the Lsass.exe process. While conversion is in progress, silo monitors that have registered and started their callbacks will now receive a notification, such that they can add their own silo context data.

The final step, then, is to “boot up” the server silo by initializing a new service session for it. You can think of this as session 0, but for the server silo. This is done through an ALPC message sent to Smss `SmApiPort`, which contains a handle to the job object created by Vmcompute, which has now become a server silo job object. Just like when creating a real user session, Smss will clone a copy of itself, except this time, the clone

will be associated with the job object at creation time. This will attach this new Smss copy to all the containerized elements of the server silo. Smss will believe this is session 0, and will perform its usual duties, such as launching Csrss.exe, Wininit.exe, Lsass.exe, etc. The "boot-up" process will continue as normal, with Wininit.exe then launching the Service Control Manager (Services.exe), which will then launch all the automatic start services, and so on. New applications can now execute in the server silo, which will run with a logon session associated with a virtual service account LUID, as described earlier.

Ancillary functionality

You may have noticed that the short description we've seen so far would obviously not result in this "boot" process actually succeeding. For example, as part of its initialization, it will want to create a named pipe called ntsvcs, which will require communicating with \Device\NamedPipe, or as Services.exe sees it, \Silos\JID\Device\NamedPipe. But no such device object exists!

As such, in order for device driver access to function, drivers must be enlightened and register their own silo monitors, which will then use the notifications to create their own per-silo device objects. The kernel provides an API, `PsAttachSiloToCurrentThread` (and matching `PsDetachSiloFromCurrentThread`), which temporarily sets the `Silo` field of the `ETHREAD` object to the passed-in job object. This will cause all access, such as that to the object manager, to be treated as if it were coming from the silo. The named pipe driver, for example, can use this functionality to then create a `NamedPipe` object under the \Device namespace, which will now be part of \Silos\JID\.

Another question is this: If applications launch in essentially a "service" session, how can they be interactive and process input and output? First, it is important to note that there is no GUI possible or permitted when launching under a Windows container, and attempting to use Remote Desktop (RDP) to access a container will also be impossible. As such, only command-line applications can execute. But even such applications normally need an "interactive" session. So how can *those* function? The secret lies in a special host process, CExecSvc.exe, which implements the container execution service. This service uses a named pipe to communicate with the Docker and Vmcompute services on the host, and is used to launch the actual containerized applications in the session. It is also used to emulate the console functionality that is normally provided by

Conhost.exe, piping the input and output through the named pipe to the actual command prompt (or PowerShell) window that was used in the first place to execute the `docker` command on the host. This service is also used when using commands such as `docker cp` to transfer files from or to the container.

Container template

Even if we take into account all the device objects that can be created by drivers as silos are created, there are still countless *other* objects, created by the kernel as well as other components, with which services running in session 0 are expected to communicate, and vice-versa. In user mode, there is no silo monitor system that would somehow allow components to support this need, and forcing every driver to always create a specialized device object to represent each silo wouldn't make sense.

If a silo wants to play music on the sound card, it shouldn't have to use a separate device object to represent the exact same sound card as every other silo would access, as well as the host itself. This would only be needed if, say, per-silo object sound isolation was required. Another example is AFD. Although it does use a silo monitor, this is to identify which user-mode service hosts the DNS client that it needs to talk to service kernel-mode DNS requests, which will be per-silo, and not to create separate \Silos\JID\Device\Afd objects, as there is a single network/Winsock stack in the system.

Beyond drivers and objects, the registry also contains various pieces of global information that must be visible and exist across all silos, which the VReg component can then provide sandboxing around.

To support all these needs, the silo namespace, registry, and file system are defined by a specialized container template file, which is located in %SystemRoot%\System32\Containers\wsc.def by default, once the Windows Containers feature is enabled in the Add/Remove Windows Features dialog box. This file describes the object manager and registry namespace and rules surrounding it, allowing the definition of symbolic links as needed to the true objects on the host. It also describes which job object, volume mount points, and network isolation policies should be used. In theory, future uses of silo objects in the Windows operating system could allow different template files to be used to provide other kinds of containerized environments. The following is an excerpt from wsc.def on a system for which containers are enabled:

Click here to view code image

```
<!-- This is a silo definition file for cmdserver.exe -->
<container>
    <namespace>
        <ob shadow="false">
            <symlink name="FileSystem" path="\FileSystem" scope="Global" />
            <symlink name="PdcPort" path="\PdcPort" scope="Global" />
            <symlink name="SeRmCommandPort" path="\SeRmCommandPort" scope="Global"
            <symlink name="Registry" path="\Registry" scope="Global" />
            <symlink name="Driver" path="\Driver" scope="Global" />
            <objdir name="BaseNamedObjects" clonesd="\BaseNamedObjects" shadow="fal
            <objdir name="GLOBAL??" clonesd="\GLOBAL??" shadow="false">
                <!-- Needed to map directories from the host -->
                <symlink name="ContainerMappedDirectories" path="\
ContainerMappedDirectories" scope="Local" />

                <!-- Valid links to \Device -->
                <symlink name="WMIDataDevice" path="\Device\WMIDataDevice" scope="L
/>
                <symlink name="UNC" path="\Device\Mup" scope="Local" />
...
            </objdir>
            <objdir name="Device" clonesd="\Device" shadow="false">
                <symlink name="Afd" path="\Device\Afd" scope="Global" />
                <symlink name="ahcache" path="\Device\ahcache" scope="Global" />
                <symlink name="CNG" path="\Device\CNG" scope="Global" />
                <symlink name="ConDrv" path="\Device\ConDrv" scope="Global" />
...
        <registry>
            <load
                key="$SiloHivesRoot$\Silo$TopLayerName$Software_Base"
                path="$TopLayerPath$\Hives\Software_Base"
                ReadOnly="true"
                />
...
                <mkkey
                    name="ControlSet001"
                    clonesd="\REGISTRY\Machine\SYSTEM\ControlSet001"
                    />
                <mkkey
                    name="ControlSet001\Control"
                    clonesd="\REGISTRY\Machine\SYSTEM\ControlSet001\Control"
                    />
```

Conclusion

This chapter examined the structure of processes, including the way processes are created and destroyed. We've seen how jobs can be used to manage a group of processes as a unit and how server silos can be used to usher in a new era of container support to Windows Server versions. The next chapter delves into threads—their structure and operation, how they're scheduled for execution, and the various ways they can be manipulated and used.